AESTHETIC AI STUDIO • VERSION 1.0

# From Ideas to Extraordinary Presentations

*User Manual and Detailed Documentation — Code, Diagrams, Tables and Figures*

A local AI presentation generator that turns a title, instructions, documents or data into designed, fully editable presentations — with real, validated PowerPoint, PDF and HTML exports.

**Creator: Dr. Asif Ahmad**

Professor, Food Science & Technology

Application: Aesthetic AI Studio v1 (31 files)

Runs locally at http://127.0.0.1:4321

Date: 19 October 2026

# Contents


1. Overview
2. User Manual — Installation, Creating, Editing and Exporting
3. How It Works — Architecture and Generation Pipeline
4. Diagrams and Figures
5. Reference Tables — Themes, Layouts, Exports, Tests and Roadmap
6. Acceptance Demo — The 30-Slide Food Safety Deck
7. Benefits, Limitations and Uniqueness
8. Security, Privacy and Deployment
9. Appendix A — Complete Source Code (6 core modules, verbatim with line numbers)
10. Appendix B — Glossary and FAQ


How to read this manual: Chapters 1–2 are for everyday users and assume no programming knowledge. Chapters 3–5 are the technical documentation. Appendix A reproduces the complete source of the six core modules exactly as shipped, so this document doubles as the code record.

# 1. Overview

**Aesthetic AI Studio v1** is a local web application — a Python/Flask backend with a browser user interface — that converts a simple title, detailed instructions, an outline, documents or raw data into a designed, editable presentation. You describe what you want; the Studio drafts an outline for your approval, builds every slide in a professional theme, lets you edit slide by slide, and exports real files you can download and keep editing elsewhere.

> **The one-sentence version:** Type a topic, approve the outline, edit the slides, download a genuine editable PowerPoint file — no account, no subscription, and no API key required.

## 1.1 What v1 is — and what it is not

v1 is a *working vertical slice* of the larger 29-section vision: prompt input, outline generation and approval, slide rendering, a three-panel editor, autosave, and verified PPTX / PDF / HTML export, all running end-to-end. The wider vision — a 10-agent orchestration team, a template marketplace, collaboration, animations and additional export formats — is the phased roadmap in Table 5, not part of v1. This manual never claims a roadmap item as a shipped feature.

### 1.2 At a glance

| Item | Detail |
|---|---|
| Creator | Dr. Asif Ahmad |
| Version | 1.0 — local foundation release |
| Delivery | 31 files; Flask backend + JavaScript frontend; one-click Windows launcher |
| Address | http://127.0.0.1:4321 (your own computer only) |
| Themes / Layouts | 6 curated themes / 13 layout patterns |
| AI providers | Built-in offline composer (no key) + optional OpenAI, Gemini and Ollama |
| Exports | PPTX (native editable objects), PDF, standalone HTML — each self-validated |
| Tests | 18 passed, 1 skipped (see Table 4) |
| Sample | 30-slide Food Safety and Hygiene deck, Dark Green & Gold theme, one click |
| Storage | Plain JSON projects under `data/` on your computer |

### 1.3 Who it is for

- **Teachers and trainers** who need a structured workshop deck quickly.
- **Researchers and academics** preparing lectures, seminars and conference talks — including Food Science and Nutrition.
- **Officers and professionals** who need a clean briefing deck without design software.
- **Anyone offline or privacy-conscious** — the built-in composer works with no internet and no key.

## 2. User Manual

### 2.1 Installation (Windows, beginner-friendly)

**Step 1 — Install Python once.** Go to python.org/downloads, download Python 3.10 or newer, run the installer and — on the first screen — tick *"Add python.exe to PATH"* before clicking Install Now. Without that tick, the launcher cannot find Python.

**Step 2 — Unzip the app.** Right-click the ZIP, choose Extract All, and move the extracted folder somewhere permanent, for example `Documents\Aesthetic-AI-Studio`. Do not run it from inside the ZIP.

**Step 3 — Start.** Double-click `Start-Windows.cmd`. First run creates a private Python environment inside the app folder and installs the pinned packages (Flask, python-pptx, reportlab, pypdf, requests) — this takes 1–3 minutes and needs internet once. Your browser then opens http://127.0.0.1:4321 automatically; if it does not, type that address yourself.

**Step 4 — Stop correctly.** Keep the black launcher window open while you work. To stop, press Ctrl+C in that window or close it. On macOS / Linux, run `bash start.sh` instead.

> **Nothing is installed system-wide.** The app's Python environment lives in its own `venv/` folder. To uninstall, delete the app folder (and Python itself only if you do not need it elsewhere).

## 2.2 Creating a presentation

1. Click **New Presentation**.
2. **Title** — the subject, e.g. “Food Safety and Hygiene Management in Hospital Canteens”.
3. **Detailed instructions** — goals, context, must-cover points, examples and style wishes. Longer, clearer instructions produce a better outline.
4. **Purpose** — choose the closest fit: academic lecture, scientific presentation, research conference, business meeting, executive or government briefing, training workshop, marketing, investor pitch, university lecture, thesis defence, sales, medical presentation, **Food Science and Nutrition**, project proposal, product launch, school presentation, professional seminar, creative storytelling, or custom.
5. **Target audience and knowledge level** — students, experts, industry professionals, executives, policymakers, researchers or a general audience; beginner, intermediate, advanced or expert.
6. **Number of slides** — the app accepts 3 to 60 in v1 (the wider vision lists up to 100).
7. **Theme** — pick one of the six in Table 1, or keep Emerald & Ivory.
8. **AI provider** — keep *Offline* to use the built-in composer with no key, or select a provider you have configured in Settings.
9. Click **Generate Outline**.

## 2.3 The outline approval step — you stay in control

Generation never jumps straight to slides. You first see the outline: a cover first, an agenda, middle sections, key takeaways, and a thank-you slide last. Reword any title or bullet, reorder items, and only then click **Build Slides**. This approval step is deliberate: fixing a one-line outline item is far faster than fixing a finished slide.

## 2.4 The three-panel editor

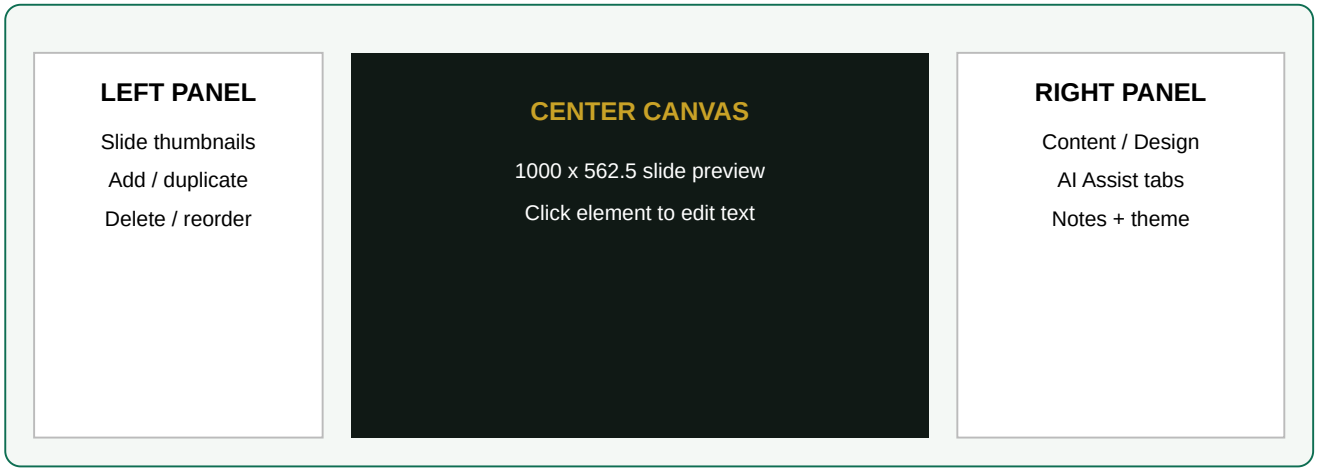



*Figure 3 — Editor layout: thumbnails and slide management on the left, the editable canvas in the centre, and Content / Design / AI-assist panels on the right.*

- **Add, duplicate, delete and reorder** slides from the left panel.
- **Edit text** by clicking an element on the canvas and typing; titles, headings, bullets, captions, tables and stat blocks are all editable text.

- **Speaker notes** — every slide carries notes; edit them in the notes area. Notes travel into the PPTX notes pane and into collapsible sections in the HTML export.
- **Theme switching** — apply a different theme to the whole deck at any time; text is kept and colours resolve from the new theme at render time.
- **Autosave** — projects persist as JSON under `data/projects/`; exports appear under `data/exports/`.

## 2.5 AI-assist editing

When an online or local provider is configured, two assist tasks are available per slide: **Improve** (rewrites that slide's title, bullets and notes through the provider) and **Relayout** (asks the provider to choose a better layout, then rebuilds that slide while keeping its identity). In Offline mode the editor explains plainly that AI assist needs a provider in Settings — it never pretends an AI call happened. Typical requests the workflow supports: make a slide more concise, change a section's design direction, or rebuild a crowded slide in a different layout — always scoped to the slide you selected.

## 2.6 Export Centre

Choose **PPTX, PDF or HTML** and click Export. The Studio builds the file, then *validates it before offering the download* and shows the validation report (green checks mean the file was re-opened and verified — see Table 3 and Figure 4). If a check fails, the report says exactly what failed; fix that slide and export again. Never treat a red check as a download-ready file.

## 2.7 Settings — API keys and connection test

1. Open **Settings**.
2. Choose a provider: **OpenAI** (default model gpt-4o-mini), **Gemini** (default gemini-2.0-flash) or **Ollama** (local; default base http://localhost:11434, a model name is required).
3. Paste the API key (OpenAI / Gemini) or set the model and base URL (Ollama), then Save.
4. Click **Test connection** — the app sends a tiny “Say OK” probe and reports the real result. A failure shows the provider's actual error, not a generic message.

**Key safety:** keys are stored only on your computer, are used server-side only, are never exposed to the browser in the public settings view, and never appear in downloaded files. The offline composer needs no key at all.

## 2.8 The one-click sample deck

Click the sample action to create the built-in 30-slide deck — “Food Safety and Hygiene Management in Hospital Canteens” in Dark Green & Gold. Use it to explore all 13 layouts, editable tables, checklists, a HACCP timeline and speaker notes before making your own deck (full showcase in Chapter 6).

### 2.9 Troubleshooting

| Problem | Fix |
|---|---|
| Launcher flashes and closes / “Python was not found” | Reinstall Python and tick “Add python.exe to PATH”. |
| Stuck installing packages | Check internet, close, re-run the launcher — it resumes. |
| Browser cannot reach the site | Confirm the launcher window is still open and port 4321 is free. |
| Export validation shows a red cross | Read the check detail, fix that slide, export again. |
| AI assist unavailable | You are in Offline mode, or the selected provider is not configured in Settings. |
| Start completely fresh | Close the app, back up then delete `data/`, restart. |

# 3. How It Works

## 3.1 The structured slide-document model

The Studio is built around one canonical document — not around PowerPoint code. A **project** holds metadata (id, title, instructions, purpose, audience, knowledge level, slide count, theme, slide size, provider, status, timestamps), an **outline**, and a list of **slides**. Each slide has a stable id, a layout id, speaker notes, and **elements**. Each element has a stable id, a type ( `text` , `shape` , `table` or `image` ), a role, and geometry on a fixed **1000 × 562.5 canvas** (16:9). `schema.py: validate_project()` rejects missing keys, unknown theme or layout ids, non-integer font sizes, invalid `#rrggbb` colours, inconsistent table rows, and any element that extends past the canvas edge. Because every renderer and exporter consumes the same validated document, an edit made once appears consistently in the preview, the PPTX, the PDF and the HTML.

## 3.2 Generation pipeline

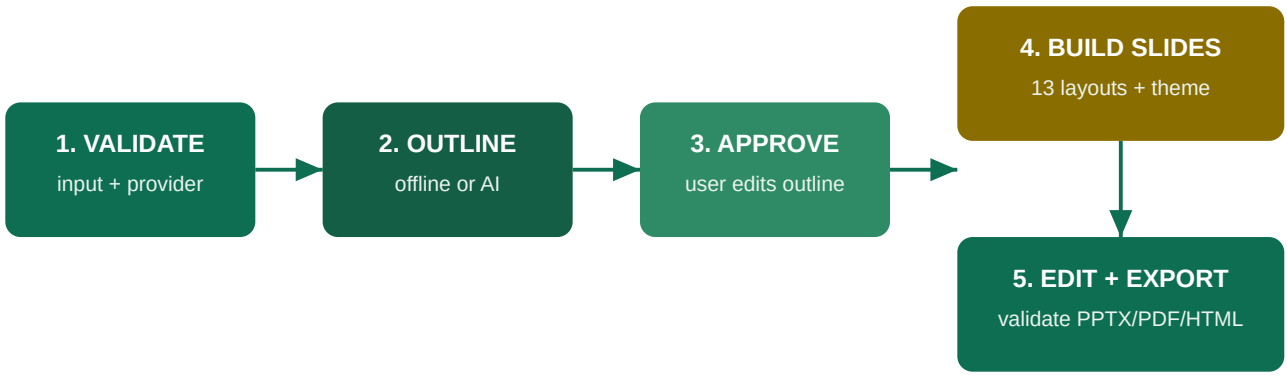



*Figure 1 — Generation pipeline: validate input, compose the outline, obtain user approval, build slides with layouts and theme, then edit and export with validation.*

1. **Validate input and provider.** Title required; slide count 3–60; theme and provider kind checked against allow-lists.
2. **Compose the outline** — offline composer, or the configured provider returning strict JSON that is coerced and checked item by item.

3. **User approval.** The outline is stored with status `outline` and shown for editing.
4. **Build slides.** `generator.py: build_slides()` maps each outline item to a layout builder, forces cover first and thank-you last, and substitutes a compatible alternate layout whenever a hint would repeat the previous slide's layout.
5. **Apply the theme and preview.** Ordinary text keeps `text_color=None` so colours resolve at render time from the current theme — which is why whole-deck theme switching works without rebuilding content.
6. **Edit, then export with validation** (Figure 4).

### 3.3 Provider-neutral AI module

`providers.py` exposes one interface — `generate_outline()`, `generate_slide_content()`, `generate_json()` and `test_connection()` — over OpenAI chat completions, Gemini generateContent, and Ollama chat. Every failure raises `ProviderError` carrying the real HTTP status, response excerpt or parse error, with a 90-second request timeout. The caller surfaces that message; the app never claims a provider responded when it did not. Markdown code fences are stripped before JSON parsing, outline items are coerced to at most six bullets, and unknown layout hints fall back to `title-content`.

### 3.4 Built-in offline composer fallback

`compose_outline_offline()` is deterministic and needs no network. It derives candidate section titles from the instructions (falling back to a five-section arc: Background & Key Concepts, Core Principles, Methods & Best Practices, Practical Applications, Challenges & Solutions), distributes the middle slides across sections, rotates through nine middle layouts, and writes audience-aware bullets and speaker notes for every slide. It is a genuine composer — useful decks with no key — not a placeholder screen.

### 3.5 Themes and layouts

Six themes each define primary, secondary, accent, background, surface, text and muted colours, a five-colour chart palette, and a heading/body font pair from a safe cross-platform set (Georgia, Trebuchet MS, Calibri, Arial). Thirteen layout builders turn outline items into positioned elements — including real tables in title-content slides, stat cards that extract the leading number from a bullet, timeline steps, comparison panels that split “A vs B” titles, and checklist bullets. See Tables 1 and 2.

### 3.6 Export adapters and validation

All three exporters share one contract: `export_*(project, theme, out_path) -> {"ok": bool, "checks": [...]}`, and `ok` is true only if *every* check passed. **PPTX** (python-pptx) maps canvas units to a 13.333 × 7.5 inch slide and creates native text boxes, shapes, connectors and tables — never slide screenshots — with notes in the notes pane. **PDF** (reportlab + pypdf) renders 960 × 540 pt landscape pages with wrapped text and table grids. **HTML** prefers the shared deck renderer and otherwise falls back to its own standalone builder with inline CSS. Image elements export as clearly labelled placeholders in v1.

# 4. Diagrams and Figures

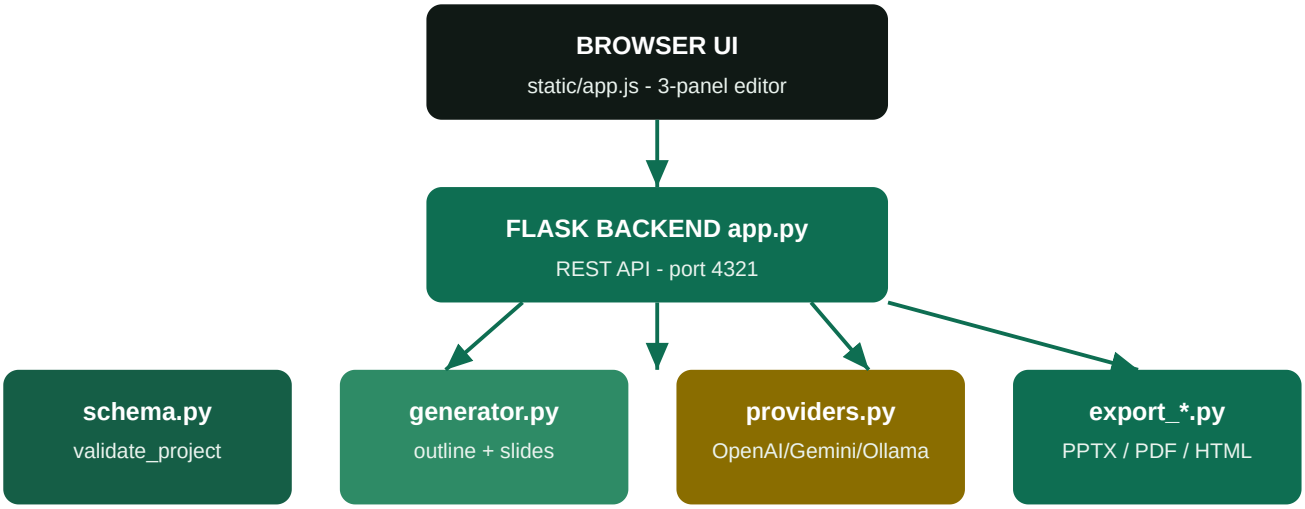



*Figure 2 — System architecture: the browser UI talks to a single Flask backend; schema, generator, providers and the three export adapters are separate modules over one validated document model. Projects persist as JSON under data/.*

Figure 1 (pipeline) appears in Section 3.2, Figure 3 (editor) in Section 2.4, and Figure 4 (export validation) below — each placed beside the workflow it explains.

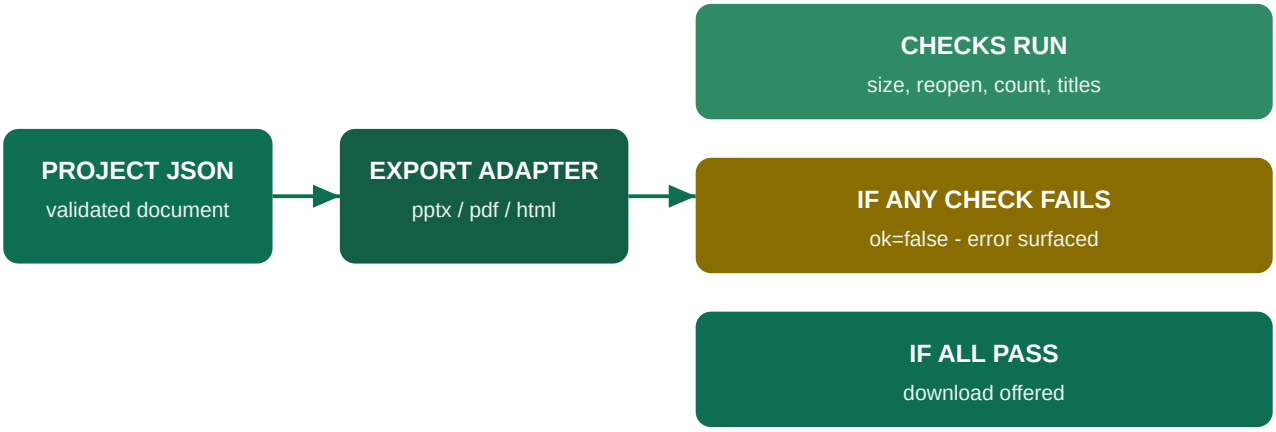



*Figure 4 — Export validation flow: an export is offered for download only when every check for that format passes; any failure is returned as ok=false with the failing check's detail.*

# 5. Reference Tables

## Table 1 — The six themes

| Theme | Primary | Accent | Background | Text | Fonts (heading / body) | Best for |
|---|---|---|---|---|---|---|
| Emerald & Ivory | #0E6E52 | #C9A227 | #FDFBF4 | #1E2A24 | Georgia / Trebuchet MS | Default; clean academic decks |
| Forest & Cream | #1B4332 | #D8B24C | #FAF6EA | #21281F | Georgia / Calibri | Warm lectures and training |
| Dark Green & Gold | #2E8B66 | #C9A227 | #101915 (dark) | #F2EFE3 | Georgia / Trebuchet MS | Executive / workshop decks; the sample deck |
| Navy & White | #1F3864 | #E8A838 | #FFFFFF | #1A2332 | Georgia / Calibri | Corporate and government briefings |
| Charcoal & Gold | #8A8F96 | #C9A227 | #232527 (dark) | #F5F2E8 | Georgia / Arial | Formal, premium presentations |
| Pastel Academic | #5E8F7B | #D9A441 | #FDF8F0 | #3A3A3A | Georgia / Calibri | Soft educational decks |

## Table 2 — The 13 layout patterns

| # | Layout id | What it builds |
|---|---|---|
| 1 | cover | Kicker, large title, subtitle, accent bar, creator credit |
| 2 | title-content | Title, accent underline, bullets; optional data table below |
| 3 | two-column | Two headed panels, bullets distributed round-robin |
| 4 | three-column | Three headed panels across the slide |
| 5 | big-quote | Large centred quotation with attribution |
| 6 | stats-dashboard | Three stat cards; value extracted from each bullet |
| 7 | timeline | Sequential step boxes (up to 7) |
| 8 | split-image-text | Image placeholder left, headed bullets right |
| 9 | section-divider | Centred section title with accent rule |
| 10 | comparison | Two contrasting panels (A vs B) |
| 11 | checklist | Checkbox-style bullet list |
| 12 | references | Numbered reference list |
| 13 | thank-you | Closing slide with discussion prompt and credit |

**Controlled variation rule:** the same layout is never used on two consecutive slides; a compatible alternate from the internal map is substituted automatically.

### Table 3 — Export formats and validation checks

| Format | Engine | Editable? | Validation checks |
|---|---|---|---|
| PPTX | python-pptx, native objects | Yes — text boxes, shapes, tables, notes | File >10 KB; re-opens; slide count matches; at least 1 shape per slide; title text present; layouts known; native text present (no screenshots) |
| PDF | reportlab + pypdf | Text selectable; pages fixed | File >10 KB; %PDF header; pypdf page count equals slide count |
| HTML | Shared renderer / standalone builder, inline CSS | Opens offline in any browser | File >5 KB; one section per slide; every slide title present |

### Table 4 — Test results (pytest)

| Suite | Covers | Result |
|---|---|---|
| test_exports.py (10 tests) | Validation-dict structure; PPTX re-open, native text, tables/bullets/notes, image placeholders; PDF page count and selectable text; HTML titles and standalone form; honest failure reporting | Passed |
| test_contract.py (7 tests) | Schema accepts valid / rejects invalid projects; offline outline shape (cover first, thank-you last); build count and layout variety; layout catalogue; all 6 themes' keys; sample deck shape (30 slides, tables, checklists, notes) | Passed |
| test_api.py (2 tests, 1 skipped in the recorded run) | Flask CRUD round-trip; export endpoints | Passed / 1 skipped |
| **Total** | | **18 passed, 1 skipped** |

### Table 5 — Roadmap: v1 versus future phases

| Capability | v1 | Future phase |
|---|---|---|
| Brief → outline → build → edit → export; 6 themes; 13 layouts; offline composer; validated PPTX/PDF/HTML | Shipped | — |
| Undo/redo, drag-resize, per-element overrides, export presets, Markdown outline import | Not in v1 | v1.1 polish |
| Provider-backed drafting with citations, brand kits, asset library, multi-language decks; real image generation; data-driven charts | Not in v1 | v2 |
| 10-agent orchestration, template marketplace, collaboration, cloud sync, animations/transitions, PPT import/redesign, ODP/MP4/GIF, handout PDFs, accessibility checker | Not in v1 | v3 / later |

## 6. Acceptance Demo — The 30-Slide Food Safety Deck

The acceptance demonstration required a professional training deck of at least 30 slides — “Food Safety and Hygiene Management in Hospital Canteens”, for senior government officers and hospital inspection teams, in a dark-green / emerald design with scientific content, hygiene checklists and suitable examples — exported as valid

editable PPTX, visually consistent PDF and functional HTML. The shipped sample (sample.py, Dark Green & Gold) meets it, and the generated instance was exported in all three formats with **all validations ok**.

| # | Slide |
|---|---|
| 1 | Cover — Food Safety and Hygiene Management in Hospital Canteens |
| 2 | Learning Objectives |
| 3 | Why Food Safety Matters in Hospitals |
| 4 | Legal & Regulatory Requirements |
| 5 | Key Food Safety Risks |
| 6 | The Temperature Danger Zone (with table) |
| 7 | Personal Hygiene Checklist |
| 8 | Cleaning vs Sanitising |
| 9 | Pest Control |
| 10 | Receiving & Storage |
| 11 | Safe Food Preparation |
| 12 | Safe Cooking Temperatures (with table) |
| 13 | Cooling & Reheating |
| 14 | Serving & Holding |
| 15 | Allergen Management |
| 16 | Water & Ice Safety |
| 17 | Waste Management |
| 18 | HACCP: The 7 Principles (timeline) |
| 19 | Daily Inspection Checklist (with table) |
| 20 | Documentation & Records |
| 21 | Staff Training |
| 22 | Building a Food Safety Culture (big quote) |
| 23 | Outbreak Response Plan (timeline) |
| 24 | Case Example: The Rice That Wasn't Cooled |
| 25 | Common Violations to Avoid |
| 26 | Corrective Actions |
| 27 | Preparing for an Audit |
| 28 | Key Takeaways |
| 29 | References & Further Reading |
| 30 | Thank You |

The deck demonstrates the features that matter in a real training session: WHO statistics on a stats-dashboard, editable temperature tables (danger zone 5–60 °C; poultry 74 °C), a personal-hygiene checklist, a HACCP 7-

principles timeline, a daily inspection checklist table, a big-quote culture slide, a worked outbreak case example, references, and speaker notes on every slide.

# 7. Benefits, Limitations and Uniqueness

## 7.1 Benefits

- **Works with no API key and no internet** after first install — the offline composer produces complete decks.
- **Real editable PowerPoint** — native text boxes, shapes and tables you can keep editing in PowerPoint, not screenshots of slides.
- **Local-first privacy** — projects, keys and exports stay on your computer; nothing is sent anywhere unless you configure and choose an online provider.
- **You approve before anything is built** — the outline step keeps control with the author.
- **Honest exports** — every file is re-opened and checked; failures are reported, never hidden.
- **Beginner-friendly** — one-click launcher, a 60-second workflow, and a full sample deck to learn from.

## 7.2 Limitations — stated plainly

- **Images are placeholders in v1.** There is no AI image generation and no stock-photo library; image elements export as labelled placeholder boxes.
- **No data-driven charts yet.** The stats-dashboard is styled stat blocks, not live charts from a dataset; build detailed charts in PowerPoint after export.
- **No presentation import.** v1 creates new decks only; it cannot open an existing PPT/PPTX and redesign it.
- **No multi-agent orchestration.** v1 runs one deterministic pipeline, not the 10-agent team of the full vision.
- **Also not in v1:** template marketplace, collaboration or cloud accounts, animations/transitions, ODP/MP4/GIF exports, speaker-notes handout PDFs, and an accessibility checker. Slide counts are 3–60 and the canvas is 16:9 only.
- **Offline content is templated.** The composer writes sound structural bullets and notes, but it does not research your topic — review and enrich the text, or configure a provider for AI drafting.

## 7.3 Uniqueness

- **Offline composer fallback as a first-class engine** — most generators stop working without a key; the Studio degrades to a deterministic composer instead of an error screen.
- **Per-check export validation** — each format returns a named checklist (re-open, slide count, native text, titles) and refuses to claim success on any failure.
- **One validated document, four surfaces** — editor preview, PPTX, PDF and HTML all render the same stable-ID document, so theme switching and edits stay consistent everywhere.
- **Layout-variety guarantee** — consecutive slides never repeat a layout, enforced in code, not left to chance.

# 8. Security, Privacy and Deployment

- Binds to 127.0.0.1 only — not exposed to your network by default.
- API keys live server-side in the local settings store; the public settings API returns only configured flags, model and base URL — never keys.

- Upload-free design in v1: briefs are typed or pasted, so there is no untrusted-file pipeline to secure yet; import security belongs to the phase that adds imports.
- Requirements: Python 3.10+, pinned dependencies in requirements.txt (Flask 3.1.0, python-pptx 1.0.2, reportlab 4.2.0, pypdf 5.1.0, requests 2.32.3, pytest 8.3.4). Internet is needed only for the first package install and for any online provider you choose.
- Backup is a folder copy: everything that matters is under `data/`.

File inventory (source delivery): app.py, generator.py, providers.py, schema.py, themes.py, sample.py, store.py, render.py, export_pptx.py, export_pdf.py, export_html.py, static/index.html, static/app.js, static/style.css, tests (conftest, test_api, test_contract, test_exports), README.md, EASY-SETUP.md, ARCHITECTURE.md, ROADMAP.md, Start-Windows.cmd, start.sh and requirements.txt — 25 source files, 31 files in the delivered package including runtime scaffolding. Appendix A reproduces the six core modules in full.

# Appendix A — Complete Source Code

Verbatim source, with line numbers, of the six core modules as shipped in v1. The remaining modules (store.py, render.py and the three exporters) are documented in Chapters 3 and 5 and in the app's own ARCHITECTURE.md.

**app.py — 510 lines**

```
"""Flask backend for Aesthetic AI Studio v1.

Run locally:  python app.py   ->  http://127.0.0.1:4321

All API routes return JSON (unless noted). Exporter modules
(export_pptx / export_pdf / export_html) are written by another agent and
imported lazily inside the export route, so the app works without them.
"""

from __future__ import annotations

import importlib
import re
from pathlib import Path

from flask import Flask, jsonify, request, send_file, send_from_directory

import providers
from generator import (
    LAYOUTS,
    apply_theme,
    build_slides,
    compose_outline_offline,
)
from render import render_slide_html
from sample import build_sample_project
from schema import blank_project, validate_project
from store import ProjectStore, SettingsStore
from themes import THEMES, get_theme, list_themes

BASE_DIR = Path(__file__).resolve().parent

_OFFLINE_ASSIST_MSG = (
    "No AI provider configured — connect one in Settings to use AI assist."
)


# ---------------------------------------------------------------------------
# helpers
# ---------------------------------------------------------------------------

def _json_body(required: bool = True):
    """Return the parsed JSON body dict, or (None, error_response)."""
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        if required:
            return None, (jsonify({"error": "Request body must be JSON"}), 400)
        return {}, None
    return data, None


def _project_cfg(project: dict, settings: SettingsStore) -> dict:
    """Build a provider cfg from the project's provider kind + stored settings."""
    kind = ((project.get("provider") or {}).get("kind") or "offline").lower()
    stored = settings.get()["providers"].get(kind, {})
    return {
        "kind": kind,
        "model": (project.get("provider") or {}).get("model") or stored.get("model", ""),
        "api_key": stored.get("api_key", ""),
        "base_url": stored.get("base_url", ""),
    }


def _brief_from_project(project: dict) -> dict:
    return {
```

```
        "title": project.get("title", ""),
        "instructions": project.get("instructions", ""),
        "purpose": project.get("purpose", ""),
        "audience": project.get("audience", ""),
        "knowledge_level": project.get("knowledge_level", ""),
        "slide_count": project.get("slide_count", 10),
    }


def _slide_summary(slide: dict) -> dict:
    """Extract {title, bullets, notes} from a built slide's elements."""
    title, bullets = "", []
    for el in slide.get("elements", []):
        if el.get("type") != "text":
            continue
        if el.get("role") == "title" and not title:
            title = el.get("text", "")
        elif el.get("role") == "bullets" and not bullets:
            for line in (el.get("text", "") or "").split("\n"):
                clean = line.strip().lstrip("•-☑✓✗▸ ").strip()
                if clean:
                    bullets.append(clean)
    return {"title": title, "bullets": bullets, "notes": slide.get("notes", "")}


# ---------------------------------------------------------------------------
# app factory
# ---------------------------------------------------------------------------

def create_app(data_root: str | Path | None = None) -> Flask:
    root = Path(data_root) if data_root else BASE_DIR / "data"
    root.mkdir(parents=True, exist_ok=True)
    (root / "exports").mkdir(parents=True, exist_ok=True)

    projects = ProjectStore(root)
    settings = SettingsStore(root)

    app = Flask(
        __name__,
        static_folder=str(BASE_DIR / "static"),
        static_url_path="/static",
    )

    # -- JSON error handlers ------------------------------------------------
    @app.errorhandler(404)
    def _not_found(_e):
        return jsonify({"error": "Not found"}), 404

    @app.errorhandler(405)
    def _method_not_allowed(_e):
        return jsonify({"error": "Method not allowed"}), 405

    def _get_project(pid: str):
        doc = projects.get(pid)
        if doc is None:
            return None, (jsonify({"error": "Project not found"}), 404)
        return doc, None

    # -- projects -----------------------------------------------------------
    @app.get("/api/projects")
    def api_list_projects():
        return jsonify(projects.list())

    @app.post("/api/projects")
    def api_create_project():
        data, err = _json_body()
        if err:
            return err
        title = str(data.get("title", "")).strip()
        if not title:
```

```
            return jsonify({"error": "'title' is required"}), 400
        try:
            slide_count = int(data.get("slide_count", 10))
        except (TypeError, ValueError):
            return jsonify({"error": "'slide_count' must be an integer"}), 400
        if not 3 <= slide_count <= 60:
            return jsonify({"error": "'slide_count' must be between 3 and 60"}), 400
        theme = data.get("theme", "emerald-ivory")
        if theme not in THEMES:
            return jsonify({"error": f"Unknown theme id: {theme!r}"}), 400
        provider = data.get("provider") or {}
        kind = str(provider.get("kind", "offline")).lower()
        if kind not in ("offline", "openai", "gemini", "ollama"):
            return jsonify({"error": f"Unknown provider kind: {kind!r}"}), 400
        doc = blank_project({
            "title": title,
            "instructions": data.get("instructions", ""),
            "purpose": data.get("purpose", ""),
            "audience": data.get("audience", ""),
            "knowledge_level": data.get("knowledge_level", ""),
            "slide_count": slide_count,
            "theme": theme,
            "provider": {"kind": kind, "model": str(provider.get("model", ""))},
        })
        errors = validate_project(doc)
        if errors:
            return jsonify({"error": "; ".join(errors)}), 400
        projects.save(doc)
        return jsonify(doc), 201

    @app.get("/api/projects/<pid>")
    def api_get_project(pid):
        doc, err = _get_project(pid)
        if err:
            return err
        return jsonify(doc)

    @app.put("/api/projects/<pid>")
    def api_update_project(pid):
        doc, err = _get_project(pid)
        if err:
            return err
        data, berr = _json_body()
        if berr:
            return berr
        updatable = (
            "title", "instructions", "purpose", "audience", "knowledge_level",
            "slide_count", "theme", "provider", "outline", "slides", "status",
        )
        for key in updatable:
            if key not in data:
                continue
            if key == "provider" and isinstance(data["provider"], dict):
                merged = dict(doc.get("provider") or {})
                merged.update(data["provider"])
                doc["provider"] = merged
            elif key == "theme":
                try:
                    apply_theme(doc, data["theme"])
                except ValueError as exc:
                    return jsonify({"error": str(exc)}), 400
            else:
                doc[key] = data[key]
        errors = validate_project(doc)
        if errors:
            return jsonify({"error": "; ".join(errors)}), 400
        projects.save(doc)
        return jsonify(doc)

    @app.delete("/api/projects/<pid>")
```

```
    def api_delete_project(pid):
        if not projects.delete(pid):
            return jsonify({"error": "Project not found"}), 404
        return jsonify({"ok": True})

    # -- outline & generation ------------------------------------------------
    @app.post("/api/projects/<pid>/outline")
    def api_generate_outline(pid):
        doc, err = _get_project(pid)
        if err:
            return err
        cfg = _project_cfg(doc, settings)
        brief = _brief_from_project(doc)
        try:
            if cfg["kind"] == "offline":
                outline = compose_outline_offline(brief)
            else:
                outline = providers.generate_outline(cfg, brief)
        except providers.ProviderError as exc:
            return jsonify({"error": str(exc)}), 502
        doc["outline"] = outline
        doc["status"] = "outline"
        try:
            projects.save(doc)
        except ValueError as exc:
            return jsonify({"error": str(exc)}), 400
        return jsonify({"outline": outline})

    @app.post("/api/projects/<pid>/generate")
    def api_generate_slides(pid):
        doc, err = _get_project(pid)
        if err:
            return err
        data, berr = _json_body(required=False)
        if berr:
            return berr
        outline = (data or {}).get("outline") or doc.get("outline") or []
        if not outline:
            return jsonify({"error": "No outline available — generate an outline first"}), 400
        doc["outline"] = outline
        try:
            theme = get_theme(doc.get("theme"))
            doc["slides"] = build_slides(doc, outline, theme)
        except Exception as exc:  # defensive: never return a half-built deck
            return jsonify({"error": f"Slide generation failed: {exc}"}), 500
        doc["status"] = "ready"
        try:
            projects.save(doc)
        except ValueError as exc:
            return jsonify({"error": str(exc)}), 400
        return jsonify(doc)

    @app.get("/api/projects/<pid>/slides/<sid>/html")
    def api_slide_html(pid, sid):
        doc, err = _get_project(pid)
        if err:
            return err
        slide = next((s for s in doc.get("slides", []) if s.get("id") == sid), None)
        if slide is None:
            return jsonify({"error": "Slide not found"}), 404
        theme = get_theme(doc.get("theme"))
        return jsonify({"html": render_slide_html(slide, theme)})

    # -- export (exporter modules written by another agent; lazy import) -----
    @app.post("/api/projects/<pid>/export")
    def api_export(pid):
        doc, err = _get_project(pid)
        if err:
            return err
        data, berr = _json_body()
```

```
        if berr:
            return berr
        fmt = str(data.get("format", "")).lower()
        if fmt not in ("pptx", "pdf", "html"):
            return jsonify({"error": "'format' must be one of: pptx, pdf, html"}), 400

        module_name = f"export_{fmt}"
        try:
            module = importlib.import_module(module_name)
        except ImportError:
            return jsonify({"error": "exporter not installed"}), 501
        export_fn = getattr(module, module_name, None) or getattr(module, "export", None)
        if not callable(export_fn):
            return jsonify({"error": f"exporter module '{module_name}' has no export function"}), 5
00

        out_dir = root / "exports" / doc["id"]
        out_dir.mkdir(parents=True, exist_ok=True)
        slug = re.sub(r"[^a-z0-9]+", "-", (doc.get("title") or "deck").lower()).strip("-")[:50] or
"deck"
        filename = f"{slug}-{doc['id']}.{fmt}"
        out_file = out_dir / filename
        theme = get_theme(doc.get("theme"))
        # Exporter convention: fn(project, theme, out_file_path) -> {"ok", "checks"}.
        try:
            try:
                result = export_fn(doc, theme, str(out_file))
            except TypeError:
                # Fall back to a simpler (project, out_dir) signature.
                result = export_fn(doc, str(out_dir))
        except Exception as exc:
            return jsonify({"error": f"Export failed: {exc}"}), 500

        validation = result if isinstance(result, dict) else {}
        if not out_file.exists():
            detail = ""
            if isinstance(validation, dict) and not validation.get("ok", True):
                failed = [c for c in validation.get("checks", []) if not c.get("passed")]
                detail = "; ".join(c.get("detail", c.get("name", "")) for c in failed[:3])
            return jsonify({"error": f"Exporter did not produce the file: {filename}"
                                     + (f" ({detail})" if detail else "")}), 500

        return jsonify({
            "ok": True,
            "filename": filename,
            "validation": validation,
            "download_url": f"/api/download/{doc['id']}/{fmt}",
        })

    @app.get("/api/download/<pid>/<fmt>")
    def api_download(pid, fmt):
        fmt = fmt.lower()
        if fmt not in ("pptx", "pdf", "html"):
            return jsonify({"error": "Unknown format"}), 404
        out_dir = root / "exports" / pid
        if not out_dir.is_dir():
            return jsonify({"error": "No exports found for this project"}), 404
        candidates = sorted(
            out_dir.glob(f"*.{fmt}"), key=lambda p: p.stat().st_mtime, reverse=True
        )
        if not candidates:
            return jsonify({"error": "No exports found for this project"}), 404
        return send_file(candidates[0], as_attachment=True,
                         download_name=candidates[0].name)

    # -- themes & layouts ----------------------------------------------------
    @app.get("/api/themes")
    def api_themes():
        return jsonify(list_themes())

```

```
    @app.get("/api/layouts")
    def api_layouts():
        return jsonify([{"id": lid, "name": l["name"]} for lid, l in LAYOUTS.items()])

    # -- settings ------------------------------------------------------------
    @app.get("/api/settings")
    def api_get_settings():
        # Public view only: configured flags + base_url/model, never api keys.
        return jsonify(settings.public_view())

    @app.post("/api/settings")
    def api_set_settings():
        data, err = _json_body()
        if err:
            return err
        provider = str(data.get("provider", "")).lower()
        if provider not in ("openai", "gemini", "ollama"):
            return jsonify({"error": "'provider' must be one of: openai, gemini, ollama"}), 400
        config = data.get("config")
        if not isinstance(config, dict):
            return jsonify({"error": "'config' must be an object"}), 400
        try:
            entry = settings.set_provider(provider, config)
        except ValueError as exc:
            return jsonify({"error": str(exc)}), 400
        return jsonify({"ok": True, "configured": bool(entry.get("configured"))})

    @app.post("/api/settings/test")
    def api_test_settings():
        data, err = _json_body()
        if err:
            return err
        provider = str(data.get("provider", "")).lower()
        if provider not in ("openai", "gemini", "ollama"):
            return jsonify({"error": "'provider' must be one of: openai, gemini, ollama"}), 400
        stored = settings.get()["providers"].get(provider, {})
        if provider == "offline" or not stored.get("configured"):
            return jsonify({"ok": False,
                            "error": f"'{provider}' provider is not configured — add credentials in
Settings."})
        cfg = {
            "kind": provider,
            "model": stored.get("model", ""),
            "api_key": stored.get("api_key", ""),
            "base_url": stored.get("base_url", ""),
        }
        try:
            providers.test_connection(cfg)
        except providers.ProviderError as exc:
            return jsonify({"ok": False, "error": str(exc)})
        return jsonify({"ok": True})

    # -- AI assist -----------------------------------------------------------
    @app.post("/api/assist")
    def api_assist():
        data, err = _json_body()
        if err:
            return err
        project_id = data.get("project_id")
        slide_id = data.get("slide_id")
        task = data.get("task")
        instruction = data.get("instruction", "")
        if not project_id or not slide_id:
            return jsonify({"error": "'project_id' and 'slide_id' are required"}), 400
        if task not in ("improve", "relayout"):
            return jsonify({"error": "'task' must be 'improve' or 'relayout'"}), 400

        doc, perr = _get_project(project_id)
        if perr:
            return perr
```

```
        cfg = _project_cfg(doc, settings)
        if cfg["kind"] == "offline":
            return jsonify({"error": _OFFLINE_ASSIST_MSG}), 400
        stored = settings.get()["providers"].get(cfg["kind"], {})
        if not stored.get("configured"):
            return jsonify({"error": f"'{cfg['kind']}' provider is not configured — add credentials in Settings."}), 400

        slide = next((s for s in doc.get("slides", []) if s.get("id") == slide_id), None)
        if slide is None:
            return jsonify({"error": "Slide not found"}), 404
        summary = _slide_summary(slide)
        brief = _brief_from_project(doc)

        try:
            if task == "improve":
                content = providers.generate_slide_content(cfg, brief, {
                    "title": summary["title"],
                    "bullets": summary["bullets"],
                    "notes": summary["notes"],
                    "instruction": instruction,
                })
                for el in slide.get("elements", []):
                    if el.get("type") == "text" and el.get("role") == "title":
                        el["text"] = content["title"]
                        break
                for el in slide.get("elements", []):
                    if el.get("type") == "text" and el.get("role") == "bullets":
                        el["text"] = "\n".join(f"• {b}" for b in content["bullets"])
                        break
                slide["notes"] = content["notes"]
            else:  # relayout — ask the provider to pick the best layout, then rebuild
                hint_list = ", ".join(sorted(LAYOUTS))
                picked = providers.generate_json(
                    cfg,
                    "You are a presentation designer. Return ONLY valid JSON — no markdown.",
                    f"Slide title: {summary['title']}\nBullets:\n"
                    + "".join(f"- {b}\n" for b in summary["bullets"])
                    + (f"\nUser instruction: {instruction}\n" if instruction else "")
                    + f"\nChoose the best layout_hint for this slide from: {hint_list}. "
                      'Return {"layout_hint": "<id>"}.',
                )
                hint = picked.get("layout_hint") if isinstance(picked, dict) else None
                if hint not in LAYOUTS or hint == slide.get("layout"):
                    from generator import _ALT_LAYOUT  # internal fallback map
                    hint = _ALT_LAYOUT.get(slide.get("layout"), "title-content")
                rebuilt = build_slides(
                    doc,
                    [{
                        "title": summary["title"] or "Untitled",
                        "bullets": summary["bullets"],
                        "layout_hint": hint,
                        "notes": summary["notes"],
                    }],
                    get_theme(doc.get("theme")),
                )[0]
                rebuilt["id"] = slide["id"]  # keep the slide's identity
                idx = doc["slides"].index(slide)
                doc["slides"][idx] = rebuilt
                slide = rebuilt
        except providers.ProviderError as exc:
            return jsonify({"error": str(exc)}), 502

        try:
            projects.save(doc)
        except ValueError as exc:
            return jsonify({"error": str(exc)}), 400
        return jsonify({"slide": slide})

    # -- sample ------------------------------------------------------------
```

```
    @app.post("/api/sample")
    def api_sample():
        doc = build_sample_project()
        projects.save(doc)
        return jsonify({"id": doc["id"]}), 201

    # -- frontend ---------------------------------------------------------
    @app.get("/")
    def index():
        index_path = BASE_DIR / "static" / "index.html"
        if index_path.exists():
            return send_from_directory(str(BASE_DIR / "static"), "index.html")
        # Placeholder until the frontend agent ships index.html.
        return (
            "<html><head><title>Aesthetic AI Studio</title></head>"
            "<body style='font-family:sans-serif;padding:40px'>"
            "<h1>Aesthetic AI Studio v1</h1>"
            "<p>Backend is running. The frontend (static/index.html) is not installed yet.</p>"
            "<p>API: <code>/api/projects</code> · <code>/api/themes</code> · "
            "<code>/api/layouts</code> · <code>/api/settings</code></p>"
            "</body></html>"
        )

    return app


if __name__ == "__main__":
    app = create_app()
    app.run(host="127.0.0.1", port=4321)
```

## generator.py — 548 lines

```
"""Slide composition engine for Aesthetic AI Studio v1.

- LAYOUTS: the 13 supported layout ids (name + description each).
- compose_outline_offline(brief): deterministic outline builder (no AI needed).
- build_slides(project, outline, theme): turns outline items into slides with
  per-layout element builders on the 1000 x 562.5 canvas.
- apply_theme(project, theme_id): switch a project's theme id.

Rules enforced in build_slides:
- the same layout is never used on two consecutive slides (a compatible
  alternate is substituted);
- the first slide is always 'cover', the last always 'thank-you'.

Color note: builders set explicit colors only for decorative shapes (accent
bars, card fills) and semantic text roles (kickers, stat values). Ordinary
text keeps text_color=None so colors resolve at render time from the current
theme plus any element override.
"""

from __future__ import annotations

import re

from schema import CANVAS_H, new_id
from themes import get_theme

LAYOUTS: dict[str, dict] = {
    "cover": {"name": "Cover",
              "description": "Opening slide: kicker, large title, subtitle, accent bar."},
    "title-content": {"name": "Title & Content",
                      "description": "Title with accent underline plus a bullet list; "
                                     "supports an optional data table."},
    "two-column": {"name": "Two Column",
                   "description": "Title plus two headed panels side by side."},
    "three-column": {"name": "Three Column",
                     "description": "Title plus three headed panels across the slide."},
    "big-quote": {"name": "Big Quote",
                  "description": "Large centered quotation with attribution."},
    "stats-dashboard": {"name": "Stats Dashboard",
                        "description": "Three stat cards, each with a big value and a label."},
    "timeline": {"name": "Timeline",
                 "description": "Sequential milestone boxes across the slide."},
    "split-image-text": {"name": "Split Image & Text",
                         "description": "Image placeholder on the left, headed bullets on the righ
t."},
    "section-divider": {"name": "Section Divider",
                        "description": "Centered section title used to open a new part."},
    "comparison": {"name": "Comparison",
                   "description": "Two headed panels contrasting A vs B."},
    "checklist": {"name": "Checklist",
                  "description": "Title plus checkbox-style bullet list."},
    "references": {"name": "References",
                   "description": "Title plus a numbered reference list."},
    "thank-you": {"name": "Thank You",
                  "description": "Closing slide: big thanks, discussion prompt, creator credit."},
}

# Compatible alternate used when a layout_hint would repeat consecutively.
_ALT_LAYOUT = {
    "cover": "section-divider",
    "section-divider": "title-content",
    "title-content": "two-column",
    "two-column": "title-content",
    "three-column": "two-column",
    "big-quote": "title-content",
```

```
    "stats-dashboard": "three-column",
    "timeline": "two-column",
    "split-image-text": "two-column",
    "comparison": "two-column",
    "checklist": "title-content",
    "references": "title-content",
    "thank-you": "section-divider",
}

_CREDIT = "Created by Dr. Asif Ahmad"


# ---------------------------------------------------------------------------
# element helpers
# ---------------------------------------------------------------------------

def _el(eid: str, etype: str, x, y, w, h, **kw) -> dict:
    e = {
        "id": eid, "type": etype,
        "x": float(x), "y": float(y), "w": float(w), "h": float(h),
        "fill": None, "line": None, "text_color": None,
        "font_size": None, "bold": False, "align": "left",
    }
    e.update(kw)
    return e


def _t(role, text, x, y, w, h, size=None, bold=False, align="left", color=None) -> dict:
    return _el(new_id("e_"), "text", x, y, w, h, role=role, text=str(text),
               font_size=size, bold=bold, align=align, text_color=color)


def _shape(shape, x, y, w, h, fill=None, line=None) -> dict:
    return _el(new_id("e_"), "shape", x, y, w, h, shape=shape, fill=fill, line=line)


def _table(rows, x, y, w, h, size=18) -> dict:
    return _el(new_id("e_"), "table", x, y, w, h, rows=rows, font_size=size)


def _img(alt, prompt, x, y, w, h) -> dict:
    return _el(new_id("e_"), "image", x, y, w, h, alt=str(alt), prompt=str(prompt))


def _title_block(item: dict, th: dict, y: int = 56) -> list[dict]:
    """Standard title + accent underline used by most content layouts."""
    return [
        _t("title", item.get("title", ""), 60, y, 880, 92, size=40, bold=True),
        _shape("rect", 62, y + 100, 72, 7, fill=th["accent"]),
    ]


def _bullets_text(bullets: list[str], marker: str = "• ") -> str:
    return "\n".join(f"{marker}{b}" for b in bullets)


# ---------------------------------------------------------------------------
# per-layout builders: fn(project, item, theme) -> [elements]
# ---------------------------------------------------------------------------

def _b_cover(project, item, th) -> list[dict]:
    kicker = project.get("purpose") or project.get("audience") or "Presentation"
    subtitle = ""
    if item.get("bullets"):
        subtitle = item["bullets"][0]
    if not subtitle:
        subtitle = (project.get("instructions") or "")[:160]
    return [
        _shape("rect", 60, 176, 90, 10, fill=th["accent"]),
        _t("kicker", str(kicker).upper(), 60, 200, 880, 36,
```

```
            size=22, bold=True, color=th["accent"]),
        _t("title", item.get("title", ""), 60, 244, 880, 140, size=54, bold=True),
        _t("subtitle", subtitle, 60, 392, 880, 72, size=24, color=th["muted"]),
        _t("caption", _CREDIT, 60, 506, 880, 30, size=16,
           align="center", color=th["muted"]),
    ]


def _b_title_content(project, item, th) -> list[dict]:
    els = _title_block(item, th)
    y = 190
    bullets = item.get("bullets") or []
    table = item.get("table")
    if bullets and table:
        # Compact intro bullets so the table fits below them on the canvas.
        els.append(_t("bullets", _bullets_text(bullets), 60, y, 880, 100, size=19))
        y += 26 + len(bullets) * 30
    elif bullets:
        els.append(_t("bullets", _bullets_text(bullets), 60, y, 880, 300, size=22))
        y += 44 + len(bullets) * 36
    if table:
        rows = [table["headers"]] + table["rows"] if isinstance(table, dict) else table
        available = CANVAS_H - 20 - y
        h = max(120.0, min(320.0, available))
        els.append(_table(rows, 60, y, 880, h, size=18))
    return els


def _column_panels(item, th, columns: list[str], bullets: list[str],
                   n: int, marker: str = "• ") -> list[dict]:
    els = _title_block(item, th)
    gap = 24
    margin = 60
    w = (1000 - 2 * margin - (n - 1) * gap) / n
    y, h = 200, 320
    chunks = [bullets[i::n] for i in range(n)]  # round-robin keeps balance
    for i in range(n):
        x = margin + i * (w + gap)
        head = columns[i] if i < len(columns) else f"Part {i + 1}"
        els.append(_shape("roundrect", x, y, w, h, fill=th["surface"], line=th["muted"]))
        els.append(_t("heading", head, x + 20, y + 18, w - 40, 44,
                      size=23, bold=True, color=th["primary"]))
        if chunks[i]:
            els.append(_t("bullets", _bullets_text(chunks[i], marker),
                          x + 20, y + 70, w - 40, h - 90, size=19))
    return els


def _b_two_column(project, item, th) -> list[dict]:
    cols = item.get("columns") or ["Key Concepts", "Practical Notes"]
    return _column_panels(item, th, cols, item.get("bullets") or [], 2)


def _b_three_column(project, item, th) -> list[dict]:
    cols = item.get("columns") or ["Part 1", "Part 2", "Part 3"]
    return _column_panels(item, th, cols, item.get("bullets") or [], 3)


def _b_big_quote(project, item, th) -> list[dict]:
    bullets = item.get("bullets") or []
    quote = bullets[0] if bullets else item.get("title", "")
    attr = bullets[1] if len(bullets) > 1 else ""
    return [
        _t("quote", "\u201c", 60, 110, 880, 100, size=96,
           color=th["accent"], align="center"),
        _t("quote", quote, 130, 205, 740, 220, size=32, align="center"),
        _t("caption", attr, 130, 435, 740, 44, size=20,
           align="center", color=th["muted"]),
    ]

```

```
205
206 def _b_stats_dashboard(project, item, th) -> list[dict]:
207     els = _title_block(item, th)
208     bullets = list((item.get("bullets") or [])[:3])
209     while len(bullets) < 3:
210         bullets.append("")
211     xs = [40, 360, 680]
212     for i, b in enumerate(bullets):
213         x = xs[i]
214         match = re.search(r"\d[\d,]*\.?\d*", b)
215         value = match.group(0) if match else "\u2014"
216         low = b.lower()
217         for word in ("million", "billion", "thousand"):
218             if word in low and word not in value.lower():
219                 value += " " + word
220                 break
221         if "%" in b and "%" not in value:
222             value += "%"
223         els.append(_shape("roundrect", x, 215, 280, 250,
224                           fill=th["surface"], line=th["accent"]))
225         els.append(_t("stat-value", value, x + 20, 245, 240, 80,
226                       size=52, bold=True, align="center", color=th["primary"]))
227         if b:
228             els.append(_t("stat-label", b, x + 20, 340, 240, 105,
229                           size=18, align="center", color=th["muted"]))
230     return els
231
232
233 def _b_timeline(project, item, th) -> list[dict]:
234     els = _title_block(item, th)
235     bullets = item.get("bullets") or []
236     n = min(max(len(bullets), 1), 7)
237     steps = (bullets + [""] * n)[:n]
238     gap, margin = 16, 40
239     w = (1000 - 2 * margin - (n - 1) * gap) / n
240     y, h = 245, 235
241     for i, step in enumerate(steps):
242         x = margin + i * (w + gap)
243         els.append(_shape("roundrect", x, y, w, h, fill=th["surface"], line=th["primary"]))
244         els.append(_t("heading", f"Step {i + 1}", x + 10, y + 14, w - 20, 34,
245                       size=19, bold=True, color=th["primary"], align="center"))
246         if step:
247             els.append(_t("body", step, x + 10, y + 56, w - 20, h - 70,
248                           size=16, align="center"))
249     return els
250
251
252 def _b_split_image_text(project, item, th) -> list[dict]:
253     els = _title_block(item, th)
254     bullets = item.get("bullets") or []
255     els.append(_img(item.get("title", "Illustration"),
256                     f"Illustration for: {item.get('title', '')}",
257                     60, 210, 400, 300))
258     els.append(_t("heading", "Highlights", 505, 210, 435, 40,
259                   size=24, bold=True, color=th["primary"]))
260     if bullets:
261         els.append(_t("bullets", _bullets_text(bullets), 505, 258, 435, 252, size=20))
262     return els
263
264
265 def _b_section_divider(project, item, th) -> list[dict]:
266     return [
267         _t("kicker", "SECTION", 60, 190, 880, 40, size=22, bold=True,
268            color=th["accent"], align="center"),
269         _shape("rect", 460, 242, 80, 8, fill=th["accent"]),
270         _t("title", item.get("title", ""), 60, 262, 880, 150,
271            size=52, bold=True, align="center"),
272         _t("caption", project.get("title", ""), 60, 420, 880, 40, size=18,
273            align="center", color=th["muted"]),
274     ]
```

```
275
276
277 def _b_comparison(project, item, th) -> list[dict]:
278     title = item.get("title", "")
279     cols = item.get("columns")
280     if not cols:
281         for sep in (" vs ", " Vs ", " VS ", " and ", " & ", " / "):
282             if sep in title:
283                 cols = [p.strip() for p in title.split(sep, 1)]
284                 break
285         else:
286             cols = ["Current State", "Desired State"]
287     return _column_panels(item, th, cols, item.get("bullets") or [], 2, marker="✓ ")
288
289
290 def _b_checklist(project, item, th) -> list[dict]:
291     els = _title_block(item, th)
292     bullets = item.get("bullets") or []
293     if bullets:
294         els.append(_t("bullets", _bullets_text(bullets, marker="☑ "),
295                       60, 195, 880, 330, size=22))
296     return els
297
298
299 def _b_references(project, item, th) -> list[dict]:
300     els = _title_block(item, th)
301     bullets = item.get("bullets") or []
302     numbered = "\n".join(f"{i + 1}. {b}" for i, b in enumerate(bullets))
303     if numbered:
304         els.append(_t("body", numbered, 60, 195, 880, 330, size=18))
305     return els
306
307
308 def _b_thank_you(project, item, th) -> list[dict]:
309     return [
310         _t("title", item.get("title") or "Thank You", 60, 170, 880, 100,
311            size=64, bold=True, align="center"),
312         _t("subtitle", "Questions & Discussion", 60, 280, 880, 60,
313            size=28, align="center", color=th["muted"]),
314         _shape("rect", 460, 360, 80, 8, fill=th["accent"]),
315         _t("caption", _CREDIT, 60, 506, 880, 30, size=16,
316            align="center", color=th["muted"]),
317     ]
318
319
320 _BUILDERS = {
321     "cover": _b_cover,
322     "title-content": _b_title_content,
323     "two-column": _b_two_column,
324     "three-column": _b_three_column,
325     "big-quote": _b_big_quote,
326     "stats-dashboard": _b_stats_dashboard,
327     "timeline": _b_timeline,
328     "split-image-text": _b_split_image_text,
329     "section-divider": _b_section_divider,
330     "comparison": _b_comparison,
331     "checklist": _b_checklist,
332     "references": _b_references,
333     "thank-you": _b_thank_you,
334 }
335
336
337 def _resolve_layout(hint: str | None, prev: str | None) -> str:
338     """Normalize a layout hint and avoid repeating the previous layout."""
339     if hint not in LAYOUTS:
340         hint = "title-content"
341     if hint == prev:
342         hint = _ALT_LAYOUT.get(hint, "title-content")
343     return hint
344
```

```
345
346 # ---------------------------------------------------------------------------
347 # offline outline composer
348 # ---------------------------------------------------------------------------
349
350 _DEFAULT_SECTIONS = [
351     "Background & Key Concepts",
352     "Core Principles",
353     "Methods & Best Practices",
354     "Practical Applications",
355     "Challenges & Solutions",
356 ]
357
358 _MIDDLE_LAYOUT_ROTATION = [
359     "title-content", "two-column", "checklist", "comparison",
360     "split-image-text", "big-quote", "three-column", "stats-dashboard",
361     "timeline",
362 ]
363
364
365 def _derive_sections(instructions: str) -> list[str]:
366     """Pull candidate section titles out of the instructions text.
367
368     Falls back to a generic 5-section arc when the instructions do not
369     contain enough usable phrases.
370     """
371     candidates: list[str] = []
372     for chunk in re.split(r"[\n;]+", instructions or ""):
373         s = chunk.strip(" \t-•*––0123456789.()\"'")
374         words = s.split()
375         if (
376             2 <= len(words) <= 7
377             and len(s) < 80
378             and s[:1].isupper()
379             and not s.endswith((".", "?", "!", ":"))
380         ):
381             s = s[0].upper() + s[1:]
382             if s not in candidates:
383                 candidates.append(s)
384     if len(candidates) >= 3:
385         return candidates[:6]
386     return list(_DEFAULT_SECTIONS)
387
388
389 def _section_bullets(section: str, brief: dict) -> list[str]:
390     title = (brief.get("title") or "this presentation").strip()
391     audience = (brief.get("audience") or "the audience").strip()
392     knowledge = (brief.get("knowledge_level") or "").strip()
393     instructions = (brief.get("instructions") or "").strip()
394     focus = instructions[:120] + ("…" if len(instructions) > 120 else "")
395     level = f" at a {knowledge.lower()} level" if knowledge else ""
396     bullets = [
397         f"What {section.lower()} covers and why it matters for {title.lower()}",
398         f"Core ideas every {audience.lower()} should understand{level}",
399     ]
400     if focus:
401         bullets.append(f"How {section.lower()} connects to the session goals: {focus}")
402     else:
403         bullets.append(f"How {section.lower()} connects to the goals of this session")
404     bullets.append(f"Practical implications and next steps for {section.lower()}")
405     return bullets[:5]
406
407
408 def _section_notes(section: str, brief: dict) -> str:
409     title = brief.get("title") or "the topic"
410     audience = brief.get("audience") or "your audience"
411     purpose = brief.get("purpose") or "the session purpose"
412     return (
413         f"Introduce {section.lower()} in the context of '{title}'. "
414         f"Tailor examples to {audience}. Emphasise how this section supports "
```

```
        f"the purpose: {purpose}. Pause for a quick question before moving on."
    )


def compose_outline_offline(brief: dict) -> list[dict]:
    """Build a deterministic outline from the brief without any AI provider."""
    n = brief.get("slide_count", 10)
    try:
        n = int(n)
    except (TypeError, ValueError):
        n = 10
    n = max(4, min(60, n))

    title = (brief.get("title") or "Untitled Presentation").strip()
    instructions = (brief.get("instructions") or "").strip()
    audience = brief.get("audience") or "the audience"
    purpose = brief.get("purpose") or "this session"
    sections = _derive_sections(instructions)

    items: list[dict] = []

    # 1. Cover.
    cover_sub = instructions[:150] if instructions else purpose
    items.append({
        "title": title,
        "bullets": [cover_sub] if cover_sub else [],
        "layout_hint": "cover",
        "notes": (
            f"Welcome {audience}. State the purpose of the session: {purpose}. "
            f"Set expectations for the {n} slides ahead and invite questions at the end."
        ),
    })

    # 2. Agenda.
    items.append({
        "title": "Agenda",
        "bullets": sections[:6],
        "layout_hint": "title-content",
        "notes": (
            "Walk through the agenda so the audience knows the journey. "
            "Mention the key takeaways slide at the end."
        ),
    })

    # Middle slides distributed across sections.
    middle_count = n - 4  # cover + agenda + takeaways + thank-you
    per_section = middle_count // len(sections)
    extra = middle_count % len(sections)
    prev_hint = "title-content"
    rot_idx = 0
    for si, section in enumerate(sections):
        count = per_section + (1 if si < extra else 0)
        for k in range(count):
            slide_title = section if count == 1 else (
                section if k == 0 else f"{section}: In Practice"
            )
            hint = _MIDDLE_LAYOUT_ROTATION[rot_idx % len(_MIDDLE_LAYOUT_ROTATION)]
            rot_idx += 1
            if hint == prev_hint:
                hint = _ALT_LAYOUT.get(hint, "title-content")
            prev_hint = hint
            items.append({
                "title": slide_title,
                "bullets": _section_bullets(section, brief),
                "layout_hint": hint,
                "notes": _section_notes(section, brief),
            })

    # Second-last: key takeaways.
    take_hint = "checklist" if prev_hint != "checklist" else "title-content"
```

```
    items.append({
        "title": "Key Takeaways",
        "bullets": [
            f"The essentials of {s.lower()} to remember" for s in sections[:4]
        ],
        "layout_hint": take_hint,
        "notes": (
            "Recap the four essentials. Ask the audience to name one action "
            "they will take after this session."
        ),
    })

    # Last: thank-you.
    items.append({
        "title": "Thank You",
        "bullets": [],
        "layout_hint": "thank-you",
        "notes": "Thank the audience and open the floor for questions and discussion.",
    })

    return items


# ---------------------------------------------------------------------------
# slide building / theming
# ---------------------------------------------------------------------------

def build_slides(project: dict, outline: list[dict], theme) -> list[dict]:
    """Turn outline items into slides with per-layout elements.

    `theme` may be a theme dict or a theme id. Enforces: first slide is
    'cover', last is 'thank-you', and no two consecutive slides share a layout.
    """
    th = theme if isinstance(theme, dict) else get_theme(theme)
    slides: list[dict] = []
    prev: str | None = None
    total = len(outline)
    for idx, item in enumerate(outline):
        hint = item.get("layout_hint") or "title-content"
        if idx == 0:
            hint = "cover"
        elif idx == total - 1 and total > 1:
            hint = "thank-you"
        hint = _resolve_layout(hint, prev)
        prev = hint
        builder = _BUILDERS[hint]
        elements = builder(project, item, th)
        slides.append({
            "id": new_id("s_"),
            "layout": hint,
            "elements": elements,
            "notes": str(item.get("notes", "")),
        })
    return slides


def apply_theme(project: dict, theme_id: str) -> dict:
    """Set a project's theme id (text is kept; colors resolve at render time)."""
    from themes import THEMES  # local import, no cycle

    if theme_id not in THEMES:
        raise ValueError(f"Unknown theme id: {theme_id!r}")
    project["theme"] = theme_id
    return project
```

**providers.py — 312 lines**

```
"""AI provider integrations for Aesthetic AI Studio v1.

Supported kinds: openai, gemini, ollama (plus 'offline', which cannot call
an API and always raises ProviderError).

cfg shape: {"kind", "model", "api_key", "base_url"}
brief shape: {"title", "instructions", "purpose", "audience",
              "knowledge_level", "slide_count"}

Every failure raises ProviderError carrying the REAL error message — the
caller must surface it, never claim success falsely. No API keys are
hardcoded anywhere; credentials arrive in cfg from the SettingsStore.
"""

from __future__ import annotations

import json
import re

import requests

REQUEST_TIMEOUT = 90  # seconds, per contract

_FENCE_RE = re.compile(r"```(?:json)?\s*(.*?)\s*```", re.DOTALL)


class ProviderError(Exception):
    """Raised for any provider failure, carrying the real error message."""


# ---------------------------------------------------------------------------
# low-level HTTP helpers
# ---------------------------------------------------------------------------

def _post_json(url: str, payload: dict, headers: dict | None = None) -> dict:
    """POST JSON and return the decoded body, raising ProviderError on failure."""
    try:
        resp = requests.post(url, json=payload, headers=headers or {},
                             timeout=REQUEST_TIMEOUT)
    except requests.RequestException as exc:
        raise ProviderError(f"Request failed: {exc}") from exc
    if resp.status_code >= 400:
        body = (resp.text or "").strip()[:500]
        raise ProviderError(f"HTTP {resp.status_code} from provider: {body}")
    try:
        data = resp.json()
    except ValueError as exc:
        raise ProviderError(
            f"Provider returned non-JSON response: {(resp.text or '')[:200]}"
        ) from exc
    if not isinstance(data, dict):
        raise ProviderError("Provider returned an unexpected response shape")
    return data


def _strip_fences(text: str) -> str:
    """Remove ```json ... ``` fences some models wrap around their JSON."""
    match = _FENCE_RE.search(text or "")
    return match.group(1) if match else (text or "").strip()


def _parse_json_object(text: str, what: str) -> dict | list:
    cleaned = _strip_fences(text)
    try:
        return json.loads(cleaned)
```

```
    except json.JSONDecodeError as exc:
        raise ProviderError(
            f"Model returned invalid JSON for {what}: {exc}; "
            f"raw excerpt: {cleaned[:200]}"
        ) from exc


# ---------------------------------------------------------------------------
# per-provider raw text calls
# ---------------------------------------------------------------------------

def _call_openai(cfg: dict, messages: list[dict], as_json: bool = True) -> str:
    base_url = (cfg.get("base_url") or "https://api.openai.com/v1").rstrip("/")
    model = cfg.get("model") or "gpt-4o-mini"
    api_key = cfg.get("api_key") or ""
    if not api_key:
        raise ProviderError("OpenAI API key is not configured (see Settings)")
    payload: dict = {"model": model, "messages": messages, "temperature": 0.7}
    if as_json:
        payload["response_format"] = {"type": "json_object"}
    data = _post_json(
        f"{base_url}/chat/completions",
        payload,
        {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json"},
    )
    try:
        return data["choices"][0]["message"]["content"] or ""
    except (KeyError, IndexError, TypeError) as exc:
        raise ProviderError(
            f"Unexpected OpenAI response shape: {json.dumps(data)[:200]}"
        ) from exc


def _call_gemini(cfg: dict, prompt: str) -> str:
    model = cfg.get("model") or "gemini-2.0-flash"
    api_key = cfg.get("api_key") or ""
    if not api_key:
        raise ProviderError("Gemini API key is not configured (see Settings)")
    url = (
        "https://generativelanguage.googleapis.com/v1beta/models/"
        f"{model}:generateContent?key={api_key}"
    )
    data = _post_json(
        url,
        {"contents": [{"parts": [{"text": prompt}]}]},
        {"Content-Type": "application/json"},
    )
    feedback = (data.get("promptFeedback") or {}).get("blockReason")
    if feedback:
        raise ProviderError(f"Gemini blocked the prompt: {feedback}")
    try:
        return data["candidates"][0]["content"]["parts"][0]["text"] or ""
    except (KeyError, IndexError, TypeError) as exc:
        raise ProviderError(
            f"Unexpected Gemini response shape: {json.dumps(data)[:200]}"
        ) from exc


def _call_ollama(cfg: dict, messages: list[dict], as_json: bool = True) -> str:
    base_url = (cfg.get("base_url") or "http://localhost:11434").rstrip("/")
    model = cfg.get("model") or ""
    if not model:
        raise ProviderError("Ollama model is not configured (see Settings)")
    payload: dict = {
        "model": model,
        "messages": messages,
        "stream": False,
    }
    if as_json:
        payload["format"] = "json"
```

```
    data = _post_json(
        f"{base_url}/api/chat", payload, {"Content-Type": "application/json"}
    )
    try:
        return data["message"]["content"] or ""
    except (KeyError, TypeError) as exc:
        raise ProviderError(
            f"Unexpected Ollama response shape: {json.dumps(data)[:200]}"
        ) from exc


def _check_kind(cfg: dict) -> str:
    kind = (cfg.get("kind") or "offline").lower()
    if kind == "offline":
        raise ProviderError(
            "Offline provider cannot call an AI API — connect a provider in Settings"
        )
    if kind not in ("openai", "gemini", "ollama"):
        raise ProviderError(f"Unknown provider kind: {cfg.get('kind')!r}")
    return kind


def generate_json(cfg: dict, system_prompt: str, user_prompt: str) -> dict | list:
    """Low-level helper: run one prompt through the configured provider.

    Returns the parsed JSON (dict or list). Raises ProviderError on any failure.
    """
    kind = _check_kind(cfg)
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ]
    if kind == "openai":
        raw = _call_openai(cfg, messages)
    elif kind == "gemini":
        raw = _call_gemini(cfg, f"{system_prompt}\n\n{user_prompt}")
    else:  # ollama
        raw = _call_ollama(cfg, messages)
    return _parse_json_object(raw, "request")


# ---------------------------------------------------------------------------
# prompts
# ---------------------------------------------------------------------------

def _layout_hint_list() -> str:
    from generator import LAYOUTS  # lazy: generator does not import providers

    return ", ".join(sorted(LAYOUTS))


def _brief_text(brief: dict) -> str:
    return (
        f"Presentation title: {brief.get('title', '')}\n"
        f"Author instructions: {brief.get('instructions', '')}\n"
        f"Purpose: {brief.get('purpose', '')}\n"
        f"Audience: {brief.get('audience', '')}\n"
        f"Audience knowledge level: {brief.get('knowledge_level', '')}\n"
        f"Desired slide count: {brief.get('slide_count', '')}"
    )


_OUTLINE_SYSTEM = (
    "You are a presentation outline planner. Return ONLY valid JSON — no "
    "markdown, no commentary. The JSON must be an object with a single key "
    '"slides", whose value is a list of slide objects. Each slide object must '
    'have exactly these keys: "title" (string), "bullets" (list of 3-5 concise '
    'strings), "layout_hint" (one of: {layouts}), and "notes" (2-3 sentences of '
    "speaker notes). Vary the layout_hint values across slides."
)
```

```
206
207
208 def _coerce_outline_item(item: dict, index: int) -> dict:
209     from generator import LAYOUTS  # lazy
210
211     if not isinstance(item, dict):
212         raise ProviderError(f"Outline slide {index} is not an object")
213     title = item.get("title")
214     if not isinstance(title, str) or not title.strip():
215         raise ProviderError(f"Outline slide {index} has no valid title")
216     bullets = item.get("bullets") or []
217     if not isinstance(bullets, list):
218         raise ProviderError(f"Outline slide {index} bullets must be a list")
219     bullets = [str(b) for b in bullets][:6]
220     hint = item.get("layout_hint") or "title-content"
221     if hint not in LAYOUTS:
222         hint = "title-content"
223     notes = item.get("notes") or ""
224     return {
225         "title": title.strip(),
226         "bullets": bullets,
227         "layout_hint": hint,
228         "notes": str(notes),
229     }
230
231
232 def generate_outline(cfg: dict, brief: dict) -> list[dict]:
233     """Generate a slide outline via the configured AI provider."""
234     _check_kind(cfg)
235     system = _OUTLINE_SYSTEM.format(layouts=_layout_hint_list())
236     user = (
237         _brief_text(brief)
238         + "\n\nWrite the outline now. First slide should introduce the topic "
239         "(layout_hint 'cover'), last slide should close it (layout_hint "
240         "'thank-you')."
241     )
242     parsed = generate_json(cfg, system, user)
243     if isinstance(parsed, dict):
244         items = parsed.get("slides", parsed.get("outline"))
245     elif isinstance(parsed, list):
246         items = parsed
247     else:
248         raise ProviderError("Model returned an unexpected outline shape")
249     if not isinstance(items, list) or not items:
250         raise ProviderError("Model returned an empty outline")
251     return [_coerce_outline_item(it, i) for i, it in enumerate(items)]
252
253
254 _SLIDE_SYSTEM = (
255     "You are a slide-content writer. Return ONLY valid JSON — no markdown, no "
256     "commentary. The JSON must be an object with exactly these keys: "
257     '"title" (string), "bullets" (list of 3-5 concise strings), "notes" '
258     "(2-3 sentences of speaker notes)."
259 )
260
261
262 def generate_slide_content(cfg: dict, brief: dict, item: dict) -> dict:
263     """(Re)generate title/bullets/notes for a single slide via the AI provider."""
264     _check_kind(cfg)
265     user = (
266         _brief_text(brief)
267         + "\n\nCurrent slide draft:\n"
268         + f"Title: {item.get('title', '')}\n"
269         + "Bullets:\n"
270         + "".join(f"- {b}\n" for b in (item.get("bullets") or []))
271         + f"\nNotes: {item.get('notes', '')}\n"
272         + "\nWrite improved content for this slide now."
273     )
274     parsed = generate_json(cfg, _SLIDE_SYSTEM, user)
275     if not isinstance(parsed, dict):
```

```
        raise ProviderError("Model returned an unexpected slide-content shape")
    title = parsed.get("title")
    if not isinstance(title, str) or not title.strip():
        raise ProviderError("Model returned no valid slide title")
    bullets = parsed.get("bullets") or []
    if not isinstance(bullets, list):
        raise ProviderError("Model returned invalid slide bullets")
    return {
        "title": title.strip(),
        "bullets": [str(b) for b in bullets][:6],
        "notes": str(parsed.get("notes") or ""),
    }


def test_connection(cfg: dict) -> bool:
    """Send a tiny 'Say OK' probe through the provider.

    Returns True on success; raises ProviderError with the real error otherwise.
    """
    kind = _check_kind(cfg)
    if kind == "openai":
        raw = _call_openai(
            cfg,
            [{"role": "user", "content": "Reply with exactly: OK"}],
            as_json=False,
        )
    elif kind == "gemini":
        raw = _call_gemini(cfg, "Reply with exactly: OK")
    else:  # ollama
        raw = _call_ollama(
            cfg,
            [{"role": "user", "content": "Reply with exactly: OK"}],
            as_json=False,
        )
    if "OK" not in (raw or "").upper():
        raise ProviderError(f"Unexpected test response from provider: {raw[:120]!r}")
    return True
```

**schema.py — 281 lines**

```
"""Schema for Aesthetic AI Studio v1.

Defines the slide-document model (plain JSON dicts), validation, id
generation, and the blank-project factory.

Canvas: 1000 x 562.5 units (16:9). All element x/y/w/h use these units.
"""

from __future__ import annotations

import re
import secrets
from datetime import datetime, timezone

# Canvas geometry (16:9).
CANVAS_W = 1000.0
CANVAS_H = 562.5

HEX_COLOR = re.compile(r"^#[0-9a-fA-F]{6}$")

STATUSES = ("draft", "outline", "ready")
PROVIDER_KINDS = ("offline", "openai", "gemini", "ollama")
ELEMENT_TYPES = ("text", "shape", "table", "image")
TEXT_ROLES = (
    "title", "subtitle", "kicker", "heading", "body",
    "bullets", "quote", "caption", "stat-value", "stat-label",
)
SHAPES = ("rect", "roundrect", "oval", "line")
ALIGNS = ("left", "center", "right")


def new_id(prefix: str) -> str:
    """Return a new id like 'p_' + 8 hex chars."""
    return f"{prefix}{secrets.token_hex(4)}"


def _now_iso() -> str:
    return datetime.now(timezone.utc).isoformat()


def blank_project(params: dict | None) -> dict:
    """Build a new, empty project document from user-supplied params."""
    p = dict(params or {})
    provider = p.get("provider") or {}
    try:
        slide_count = int(p.get("slide_count", 10))
    except (TypeError, ValueError):
        slide_count = 10
    now = _now_iso()
    return {
        "id": new_id("p_"),
        "title": str(p.get("title", "Untitled Presentation")),
        "instructions": str(p.get("instructions", "")),
        "purpose": str(p.get("purpose", "")),
        "audience": str(p.get("audience", "")),
        "knowledge_level": str(p.get("knowledge_level", "")),
        "slide_count": slide_count,
        "theme": str(p.get("theme", "emerald-ivory")),
        "slide_size": "16:9",
        "provider": {
            "kind": str(provider.get("kind", "offline")),
            "model": str(provider.get("model", "")),
        },
        "status": "draft",
        "outline": [],
```

```
        "slides": [],
        "created": now,
        "updated": now,
    }


def _is_num(v) -> bool:
    return isinstance(v, (int, float)) and not isinstance(v, bool)


def _check_color(errors: list, where: str, value) -> None:
    if value is not None and not (isinstance(value, str) and HEX_COLOR.match(value)):
        errors.append(f"{where}: color must be '#rrggbb' or null, got {value!r}")


def validate_project(doc) -> list[str]:
    """Validate a project document.

    Returns a list of human-readable error strings; [] means valid.
    Checks required keys, value types, element bounds (0..1000 / 0..562.5),
    and that layout/theme ids are known.
    """
    from themes import THEMES          # lazy: themes is standalone, no cycle
    from generator import LAYOUTS      # lazy: generator imports schema (new_id)

    errors: list[str] = []
    if not isinstance(doc, dict):
        return ["project must be a JSON object"]

    required = (
        "id", "title", "instructions", "purpose", "audience",
        "knowledge_level", "slide_count", "theme", "slide_size",
        "provider", "status", "outline", "slides", "created", "updated",
    )
    for key in required:
        if key not in doc:
            errors.append(f"missing required key: {key!r}")

    def _str(key, allow_empty=True):
        if key in doc and not isinstance(doc[key], str):
            errors.append(f"{key!r} must be a string")
        elif key in doc and not allow_empty and not doc[key].strip():
            errors.append(f"{key!r} must not be empty")

    _str("id", allow_empty=False)
    _str("title", allow_empty=False)
    _str("instructions")
    _str("purpose")
    _str("audience")
    _str("knowledge_level")
    _str("created", allow_empty=False)
    _str("updated", allow_empty=False)

    if "slide_count" in doc and (
        not isinstance(doc["slide_count"], int) or isinstance(doc["slide_count"], bool)
    ):
        errors.append("'slide_count' must be an integer")

    if "theme" in doc:
        if not isinstance(doc["theme"], str):
            errors.append("'theme' must be a string")
        elif doc["theme"] not in THEMES:
            errors.append(f"unknown theme id: {doc['theme']!r}")

    if "slide_size" in doc and doc["slide_size"] != "16:9":
        errors.append("'slide_size' must be '16:9'")

    prov = doc.get("provider")
    if "provider" in doc:
        if not isinstance(prov, dict):
```

```
            errors.append("'provider' must be an object")
        else:
            if prov.get("kind") not in PROVIDER_KINDS:
                errors.append(f"unknown provider kind: {prov.get('kind')!r}")
            if "model" in prov and not isinstance(prov["model"], str):
                errors.append("'provider.model' must be a string")

    if "status" in doc and doc["status"] not in STATUSES:
        errors.append(f"unknown status: {doc['status']!r}")

    if "outline" in doc:
        if not isinstance(doc["outline"], list):
            errors.append("'outline' must be a list")
        else:
            for i, item in enumerate(doc["outline"]):
                where = f"outline[{i}]"
                if not isinstance(item, dict):
                    errors.append(f"{where} must be an object")
                    continue
                for k in ("title", "bullets", "layout_hint", "notes"):
                    if k not in item:
                        errors.append(f"{where} missing key: {k!r}")
                if "title" in item and not isinstance(item["title"], str):
                    errors.append(f"{where}.title must be a string")
                if "bullets" in item:
                    if not isinstance(item["bullets"], list) or not all(
                        isinstance(b, str) for b in item["bullets"]
                    ):
                        errors.append(f"{where}.bullets must be a list of strings")
                if "layout_hint" in item:
                    if not isinstance(item["layout_hint"], str):
                        errors.append(f"{where}.layout_hint must be a string")
                    elif item["layout_hint"] not in LAYOUTS:
                        errors.append(
                            f"{where} unknown layout_hint: {item['layout_hint']!r}"
                        )
                if "notes" in item and not isinstance(item["notes"], str):
                    errors.append(f"{where}.notes must be a string")

    if "slides" in doc:
        if not isinstance(doc["slides"], list):
            errors.append("'slides' must be a list")
        else:
            for i, slide in enumerate(doc["slides"]):
                _validate_slide(errors, f"slides[{i}]", slide, LAYOUTS)

    return errors


def _validate_slide(errors: list, where: str, slide, layouts) -> None:
    if not isinstance(slide, dict):
        errors.append(f"{where} must be an object")
        return
    for k in ("id", "layout", "elements", "notes"):
        if k not in slide:
            errors.append(f"{where} missing key: {k!r}")
    if "id" in slide and not isinstance(slide["id"], str):
        errors.append(f"{where}.id must be a string")
    if "layout" in slide:
        if not isinstance(slide["layout"], str):
            errors.append(f"{where}.layout must be a string")
        elif slide["layout"] not in layouts:
            errors.append(f"{where} unknown layout: {slide['layout']!r}")
    if "notes" in slide and not isinstance(slide["notes"], str):
        errors.append(f"{where}.notes must be a string")
    if "elements" in slide:
        if not isinstance(slide["elements"], list):
            errors.append(f"{where}.elements must be a list")
        else:
            for j, el in enumerate(slide["elements"]):
```

```
                _validate_element(errors, f"{where}.elements[{j}]", el)


def _validate_element(errors: list, where: str, el) -> None:
    if not isinstance(el, dict):
        errors.append(f"{where} must be an object")
        return
    for k in ("id", "type", "x", "y", "w", "h"):
        if k not in el:
            errors.append(f"{where} missing key: {k!r}")
    if el.get("type") not in ELEMENT_TYPES:
        errors.append(f"{where} unknown element type: {el.get('type')!r}")
        return

    for k in ("x", "y", "w", "h"):
        if k in el and not _is_num(el[k]):
            errors.append(f"{where}.{k} must be a number")

    # Bounds check on the 1000 x 562.5 canvas.
    if all(_is_num(el.get(k)) for k in ("x", "y", "w", "h")):
        x, y, w, h = (float(el[k]) for k in ("x", "y", "w", "h"))
        if not (0 <= x <= CANVAS_W):
            errors.append(f"{where}.x out of canvas bounds 0..{CANVAS_W}")
        if not (0 <= y <= CANVAS_H):
            errors.append(f"{where}.y out of canvas bounds 0..{CANVAS_H}")
        if w <= 0:
            errors.append(f"{where}.w must be > 0")
        if h <= 0:
            errors.append(f"{where}.h must be > 0")
        if x + w > CANVAS_W + 1e-6:
            errors.append(f"{where} extends past canvas right edge (x+w > {CANVAS_W})")
        if y + h > CANVAS_H + 1e-6:
            errors.append(f"{where} extends past canvas bottom edge (y+h > {CANVAS_H})")

    _check_color(errors, f"{where}.fill", el.get("fill"))
    _check_color(errors, f"{where}.line", el.get("line"))
    _check_color(errors, f"{where}.text_color", el.get("text_color"))
    if "font_size" in el and el["font_size"] is not None and (
        not isinstance(el["font_size"], int) or isinstance(el["font_size"], bool)
    ):
        errors.append(f"{where}.font_size must be an integer or null")
    if "bold" in el and not isinstance(el["bold"], bool):
        errors.append(f"{where}.bold must be a boolean")
    if "align" in el and el["align"] not in ALIGNS:
        errors.append(f"{where}.align must be one of {ALIGNS}")

    etype = el["type"]
    if etype == "text":
        if el.get("role") not in TEXT_ROLES:
            errors.append(f"{where} unknown text role: {el.get('role')!r}")
        if "text" in el and not isinstance(el["text"], str):
            errors.append(f"{where}.text must be a string")
    elif etype == "shape":
        if el.get("shape") not in SHAPES:
            errors.append(f"{where} unknown shape: {el.get('shape')!r}")
    elif etype == "table":
        rows = el.get("rows")
        if not isinstance(rows, list) or not rows:
            errors.append(f"{where}.rows must be a non-empty list of rows")
        else:
            width = None
            for r, row in enumerate(rows):
                if not isinstance(row, list) or not all(
                    isinstance(c, str) for c in row
                ):
                    errors.append(f"{where}.rows[{r}] must be a list of strings")
                    continue
                if width is None:
                    width = len(row)
                elif len(row) != width:
```

```
                    errors.append(f"{where}.rows[{r}] has inconsistent column count")
    elif etype == "image":
        if "alt" in el and not isinstance(el["alt"], str):
            errors.append(f"{where}.alt must be a string")
        if "prompt" in el and not isinstance(el["prompt"], str):
            errors.append(f"{where}.prompt must be a string")
```

**themes.py — 109 lines**

```
"""Color themes for Aesthetic AI Studio v1.

Six curated themes. Fonts are restricted to the safe cross-platform set:
Georgia, Trebuchet MS, Calibri, Arial.
"""

from __future__ import annotations

THEMES: dict[str, dict] = {
    "emerald-ivory": {
        "name": "Emerald & Ivory",
        "primary": "#0E6E52",
        "secondary": "#155E46",
        "accent": "#C9A227",
        "background": "#FDFBF4",
        "surface": "#FFFFFF",
        "text": "#1E2A24",
        "muted": "#6B7A70",
        "chart": ["#0E6E52", "#2A9D6F", "#C9A227", "#7FB69E", "#3D5A45"],
        "heading_font": "Georgia",
        "body_font": "Trebuchet MS",
    },
    "forest-cream": {
        "name": "Forest & Cream",
        "primary": "#1B4332",
        "secondary": "#2D6A4F",
        "accent": "#D8B24C",
        "background": "#FAF6EA",
        "surface": "#FFFFFF",
        "text": "#21281F",
        "muted": "#707A68",
        "chart": ["#1B4332", "#2D6A4F", "#D8B24C", "#95D5B2", "#52796F"],
        "heading_font": "Georgia",
        "body_font": "Calibri",
    },
    "dark-green-gold": {
        "name": "Dark Green & Gold",
        "primary": "#2E8B66",
        "secondary": "#1B5E43",
        "accent": "#C9A227",
        "background": "#101915",
        "surface": "#1A2B21",
        "text": "#F2EFE3",
        "muted": "#9AA79C",
        "chart": ["#C9A227", "#2E8B66", "#E8D27A", "#7FB69E", "#4C8C6B"],
        "heading_font": "Georgia",
        "body_font": "Trebuchet MS",
    },
    "navy-white": {
        "name": "Navy & White",
        "primary": "#1F3864",
        "secondary": "#2E5AAC",
        "accent": "#E8A838",
        "background": "#FFFFFF",
        "surface": "#F4F6FA",
        "text": "#1A2332",
        "muted": "#64748B",
        "chart": ["#1F3864", "#2E5AAC", "#E8A838", "#7BA7D9", "#4C6B8A"],
        "heading_font": "Georgia",
        "body_font": "Calibri",
    },
    "charcoal-gold": {
        "name": "Charcoal & Gold",
        "primary": "#8A8F96",
        "secondary": "#45484D",
```

```
        "accent": "#C9A227",
        "background": "#232527",
        "surface": "#2E3134",
        "text": "#F5F2E8",
        "muted": "#A8A49A",
        "chart": ["#C9A227", "#E8D27A", "#8A8F96", "#6B6F75", "#A8801F"],
        "heading_font": "Georgia",
        "body_font": "Arial",
    },
    "pastel-academic": {
        "name": "Pastel Academic",
        "primary": "#5E8F7B",
        "secondary": "#A8CBB8",
        "accent": "#D9A441",
        "background": "#FDF8F0",
        "surface": "#FFFFFF",
        "text": "#3A3A3A",
        "muted": "#8A8A8A",
        "chart": ["#5E8F7B", "#A8CBB8", "#D9A441", "#E8C99A", "#7FB3D5"],
        "heading_font": "Georgia",
        "body_font": "Calibri",
    },
}

_DEFAULT_THEME_ID = "emerald-ivory"


def get_theme(theme_id: str | None) -> dict:
    """Return the theme dict for an id, falling back to emerald-ivory."""
    if isinstance(theme_id, str) and theme_id in THEMES:
        return THEMES[theme_id]
    return THEMES[_DEFAULT_THEME_ID]


def list_themes() -> list[dict]:
    """Return [{id, name, swatches:[primary, accent, background]}] for all themes."""
    return [
        {
            "id": tid,
            "name": t["name"],
            "swatches": [t["primary"], t["accent"], t["background"]],
        }
        for tid, t in THEMES.items()
    ]
```

**sample.py — 510 lines**

```python
"""Sample project: a ready-made 30-slide deck.

"Food Safety and Hygiene Management in Hospital Canteens" — real curated
content (objectives, temperature tables, checklists, HACCP timeline, ...),
theme dark-green-gold, status "ready".
"""

from __future__ import annotations

from datetime import datetime, timezone

from schema import new_id
from generator import build_slides
from themes import get_theme

_TITLE = "Food Safety and Hygiene Management in Hospital Canteens"


def _S(title, bullets, layout, notes, **extra):
    """Shorthand for building an outline item."""
    item = {
        "title": title,
        "bullets": list(bullets),
        "layout_hint": layout,
        "notes": notes,
    }
    item.update(extra)
    return item


def _outline() -> list[dict]:
    return [
        _S(
            _TITLE,
            ["Protecting patients, staff, and visitors through safe food handling, "
             "from receiving to serving"],
            "cover",
            "Welcome the participants — canteen staff, food handlers, and supervisors. "
            "Explain that hospital patients are far more vulnerable to foodborne illness "
            "than the general public, so our standards must be higher than a regular "
            "restaurant. Outline the journey of the session.",
        ),
        _S(
            "Learning Objectives",
            [
                "Identify the major food safety hazards in a hospital canteen",
                "Apply temperature control rules at every stage of food handling",
                "Demonstrate correct personal hygiene and handwashing practice",
                "Follow cleaning, sanitation, and pest-control procedures",
                "Implement the 7 principles of HACCP in daily work",
                "Respond correctly to a suspected food safety incident",
            ],
            "checklist",
            "Read the objectives aloud and ask participants which one feels most "
            "relevant to their daily work. Tell them they will be able to do all six "
            "by the end of this session.",
        ),
        _S(
            "Why Food Safety Matters in Hospitals",
            [
                "600 million people fall ill from contaminated food every year (WHO)",
                "420,000 deaths worldwide each year are linked to foodborne disease",
                "1 in 10 people globally are affected by foodborne illness annually",
            ],
            "stats-dashboard",
```

```
            "These are World Health Organization estimates. Stress the third point: "
            "hospital patients — the elderly, children, post-surgery and "
            "immunocompromised patients — are far more likely to suffer severe outcomes. "
            "One outbreak can close a canteen and damage the hospital's reputation.",
        ),
        _S(
            "Legal & Regulatory Requirements",
            [
                "Canteen premises must be licensed by the provincial food authority "
                "(e.g., Punjab Food Authority) and display the licence",
                "Follow a HACCP-based food safety management system (ISO 22000 principles)",
                "All food handlers need valid medical fitness certificates, renewed annually",
                "Maintain inspection-ready records: temperature logs, cleaning schedules, "
                "supplier invoices",
                "Non-compliance can mean fines, sealing of premises, or licence suspension",
            ],
            "title-content",
            "Adapt the authority name to your province or country. Emphasise that "
            "inspectors can arrive unannounced — compliance must be daily habit, not "
            "a last-minute scramble.",
        ),
        _S(
            "Key Food Safety Risks",
            [
                "Biological hazards: Salmonella, E. coli, Listeria, and norovirus from "
                "contaminated food or handlers",
                "Chemical hazards: cleaning-agent residues, pesticide traces, unlabelled "
                "chemicals stored near food",
                "Physical hazards: glass shards, metal fragments, hair, or packaging pieces",
                "Cross-contamination: raw meat juices touching ready-to-eat foods or surfaces",
                "Time–temperature abuse: food left too long in the danger zone (5–60 °C)",
            ],
            "title-content",
            "Give one canteen example per hazard: undercooked chicken (biological), "
            "sanitiser sprayed near uncovered salad (chemical), a broken glass near the "
            "serving counter (physical). Ask staff for examples they have seen.",
        ),
        _S(
            "The Temperature Danger Zone",
            [
                "Bacteria multiply fastest between 5 °C and 60 °C (41–140 °F) — "
                "doubling every ~20 minutes",
                "Never leave perishable food in the danger zone for more than 2 hours "
                "(1 hour if ambient temperature exceeds 32 °C)",
            ],
            "title-content",
            "This is the single most important concept in the session. Draw the danger "
            "zone on a flip chart. Quiz the group: 'The fridge broke at 10 am — when "
            "must the milk be discarded?'",
            table={
                "headers": ["Stage", "Safe Temperature", "Rule"],
                "rows": [
                    ["Refrigeration", "≤ 5 °C (41 °F)", "Check twice daily with a calibrated thermo
meter"],
                    ["Danger zone", "5–60 °C (41–140 °F)", "Maximum 2 hours cumulative exposure"],
                    ["Hot holding", "≥ 60 °C (140 °F)", "Stir and check temperature hourly"],
                    ["Freezing", "≤ −18 °C (0 °F)", "Freeze fresh stock promptly; label dates"],
                ],
            },
        ),
        _S(
            "Personal Hygiene Checklist",
            [
                "Wash hands with soap for at least 20 seconds: before work, after toilet, "
                "after touching raw food, waste, or face",
                "Wear a clean uniform, apron, hairnet, and closed shoes every shift",
                "Keep nails short and clean; no nail polish or jewellery except a plain band",
                "Cover cuts and wounds with a waterproof blue bandage plus a glove",
                "Report illness — vomiting, diarrhoea, jaundice, infected skin — and stay "
                "off food-handling duty until cleared",
```

```
                "Use single-use gloves for ready-to-eat food; change them every 4 hours "
                "or when torn",
            ],
            "checklist",
            "Demonstrate the WHO 6-step handwashing technique live if a sink is nearby. "
            "Remind staff that gloves are not a substitute for handwashing.",
        ),
        _S(
            "Cleaning vs Sanitising",
            [
                "Cleaning removes visible dirt and grease with detergent and hot water",
                "Sanitising kills microbes on an already-clean surface — it cannot work "
                "through grease",
                "Always clean first, then sanitise; respect the contact time on the label",
                "Use the correct dilution — more chemical is not more effective and leaves "
                "toxic residues",
                "Colour-code cloths and mops: e.g., red for washrooms, green for food areas",
                "Clean as you go: wipe spills immediately, sanitise boards between tasks",
            ],
            "two-column",
            "Explain the two-step logic with a simple analogy: you cannot disinfect a "
            "dirty wound. Show the dilution chart posted in the chemical store.",
            columns=["Cleaning", "Sanitising"],
        ),
        _S(
            "Pest Control",
            [
                "Seal entry points: door sweeps, window screens, sealed pipe gaps",
                "Store all food 15 cm off the floor in sealed, labelled containers",
                "Remove waste daily; keep bins covered and the area around them clean",
                "Inspect insect traps and rodent stations weekly and log findings",
                "Never spray pesticides near food or food-contact surfaces yourself — "
                "call the licensed pest-control operator",
                "Report any sighting or droppings to the supervisor immediately",
            ],
            "two-column",
            "Pests carry Salmonella, E. coli, and allergens. One cockroach sighting by a "
            "patient's attendant can destroy trust in the whole hospital kitchen.",
            columns=["Prevention", "Monitoring & Action"],
        ),
        _S(
            "Receiving & Storage",
            [
                "Check delivery temperatures: chilled ≤ 5 °C, frozen ≤ −18 °C — reject "
                "anything warmer",
                "Reject damaged packaging, dented cans, expired items, or off-smell produce",
                "Apply FIFO (first in, first out) and FEFO (first expired, first out); "
                "label everything with dates",
                "Store raw meat on the lowest shelves, below cooked and ready-to-eat foods",
                "Keep dry stores cool, dry, ventilated, and 15 cm off the floor",
                "Record supplier name, item, quantity, and temperature on the receiving log",
            ],
            "title-content",
            "Walk the group through the actual receiving area after this slide if possible. "
            "Show where raw chicken is stored versus cooked rice — the separation must be "
            "visible.",
        ),
        _S(
            "Safe Food Preparation",
            [
                "Thaw frozen food in the refrigerator, under cold running water, or in the "
                "microwave — never on the countertop",
                "Use colour-coded boards and knives: red for raw meat, blue for raw fish, "
                "green for vegetables, white for cooked food",
                "Wash fruits and vegetables thoroughly under running potable water",
                "Marinate only in the refrigerator, never at room temperature",
                "Prepare raw and ready-to-eat foods at separate times or stations",
                "Limit preparation batches so food spends minimal time at room temperature",
            ],
            "title-content",
```

```
            "Ask: 'Where do you thaw chicken at home?' — then contrast home habits with "
            "canteen rules. Show the colour-coded boards the kitchen actually uses.",
        ),
        _S(
            "Safe Cooking Temperatures",
            [
                "Always verify with a calibrated food thermometer — colour is not reliable",
                "Insert the probe into the thickest part, away from bone",
                "Calibrate thermometers weekly in ice water (should read 0 °C)",
            ],
            "title-content",
            "Demonstrate with a real thermometer and a piece of chicken if the kitchen is "
            "accessible. Undercooking is the top cause of poultry-related outbreaks.",
            table={
                "headers": ["Food", "Minimum Internal Temperature"],
                "rows": [
                    ["Poultry (whole, pieces, ground)", "74 °C (165 °F)"],
                    ["Ground beef, pork, lamb", "71 °C (160 °F)"],
                    ["Fish and seafood", "63 °C (145 °F)"],
                    ["Pork chops, steaks, roasts", "63 °C (145 °F) + 3 min rest"],
                    ["Eggs for immediate service", "63 °C (145 °F)"],
                    ["Reheated / leftover foods", "74 °C (165 °F)"],
                ],
            },
        ),
        _S(
            "Cooling & Reheating",
            [
                "Cool cooked food fast: from 60 °C to 21 °C within 2 hours, then to 5 °C "
                "within 4 more hours",
                "Use shallow pans, ice baths, and stirring to speed cooling — never cool a "
                "deep pot on the counter",
                "Refrigerate cooled food promptly in covered, labelled containers",
                "Reheat only once, rapidly, to a steaming 74 °C (165 °F) throughout",
                "Discard food that stayed too long in the danger zone — when in doubt, "
                "throw it out",
            ],
            "title-content",
            "Bacillus cereus in cooled rice and Clostridium perfringens in cooling curries "
            "are classic hospital-kitchen outbreaks. The two-stage cooling rule prevents both.",
        ),
        _S(
            "Serving & Holding",
            [
                "Hold hot food at or above 60 °C and cold food at or below 5 °C — check hourly",
                "Never mix fresh batches into food already on display; use small replenishment "
                "trays",
                "Protect displayed food with sneeze guards and covers",
                "Provide separate serving utensils for each dish; never let hands touch "
                "ready-to-eat food",
                "Discard any food held in the danger zone beyond 2 hours — do not taste-test "
                "to decide",
            ],
            "title-content",
            "The serving counter is where patients and attendants judge your hygiene. "
            "A clean, guarded, well-labelled display builds confidence.",
        ),
        _S(
            "Allergen Management",
            [
                "The 9 major allergens: milk, eggs, fish, shellfish, tree nuts, peanuts, "
                "wheat, soy, and sesame",
                "Label menus and display cards clearly when dishes contain allergens",
                "Prepare allergen-free meals first, with cleaned equipment and utensils",
                "Never guess — check ingredient labels and ask the patient about allergies",
                "Train all serving staff on the emergency response for anaphylaxis",
            ],
            "comparison",
            "For hospital canteens serving patients, allergen control is clinical safety, "
            "not just preference. One undeclared peanut trace can trigger anaphylaxis.",
```

```
            columns=["The 9 Major Allergens", "Control Measures"],
        ),
        _S(
            "Water & Ice Safety",
            [
                "Use only potable water for cooking, drinking, washing produce, and making ice",
                "Clean and disinfect overhead/underground water tanks at least every 6 months",
                "If using RO/UV treatment, replace filters and service units on schedule",
                "Make ice from safe water and handle it with scoops — never with hands or glasses",
                "Test water quality periodically (microbiological + chemical) and keep reports",
            ],
            "title-content",
            "Contaminated water or ice can sicken hundreds at once. Show the latest water "
            "test report to prove the system works.",
        ),
        _S(
            "Waste Management",
            [
                "Segregate waste at source: food waste, recyclables, and general waste",
                "Use foot-operated, covered, leak-proof bins lined with bags",
                "Remove waste from food areas at least daily — more often in summer",
                "Clean and disinfect bins and the waste storage area after each emptying",
                "Keep waste storage away from food preparation and receiving areas",
                "Maintain grease traps; never pour oil down the drain",
            ],
            "title-content",
            "Overflowing bins are the fastest route to a pest infestation and a failed "
            "inspection. Assign clear ownership: who empties which bin, when.",
        ),
        _S(
            "HACCP: The 7 Principles",
            [
                "Conduct a hazard analysis for each menu item and process",
                "Determine the Critical Control Points (CCPs) — e.g., cooking, cooling",
                "Establish critical limits for each CCP (temperatures, times)",
                "Set up monitoring procedures for every CCP",
                "Define corrective actions when a limit is breached",
                "Verify the system works (calibration, audits, reviews)",
                "Keep records of everything — if it isn't written down, it didn't happen",
            ],
            "timeline",
            "Walk through one dish — e.g., chicken curry — and map its CCPs: receiving "
            "temperature, cooking to 74 °C, cooling within 6 hours, reheating to 74 °C. "
            "That makes HACCP concrete instead of abstract.",
        ),
        _S(
            "Daily Inspection Checklist",
            [
                "Complete every row before service; the supervisor signs off",
                "Any 'No' triggers an immediate corrective action — logged, not hidden",
            ],
            "title-content",
            "Print this as a laminated sheet for the kitchen wall. Inspections pass when "
            "the paperwork matches reality — never fabricate a log entry.",
            table={
                "headers": ["Area", "Check Point", "Frequency"],
                "rows": [
                    ["Personal hygiene", "Handwashing, uniforms, illness reporting", "Daily"],
                    ["Temperatures", "Fridge, freezer, and holding-unit logs", "Twice daily"],
                    ["Cleaning", "Sanitation schedule signed off", "Daily"],
                    ["Pest control", "Traps and stations inspected", "Weekly"],
                    ["Storage", "FIFO/FEFO, labelling, expiry dates", "Weekly"],
                    ["Water & waste", "Tank levels, bin emptying", "Daily"],
                ],
            },
        ),
        _S(
            "Documentation & Records",
            [
                "Temperature logs: deliveries, fridges, freezers, cooking, holding",
```

```
                "Cleaning and sanitation checklists, signed and dated",
                "Supplier records: invoices, certificates, receiving logs",
                "Staff training records: attendance, topics, assessments",
                "Corrective-action log: what went wrong, what was done, who verified",
                "Keep records for at least one year (or per local regulation)",
            ],
            "title-content",
            "Records are your proof during inspections and outbreak investigations. "
            "Show an example of a well-kept temperature log versus a blank one.",
        ),
        _S(
            "Staff Training",
            [
                "Induction training for every new handler before their first shift",
                "Annual refresher plus targeted sessions after any incident",
                "Hands-on demos: handwashing, thermometer use, allergen awareness",
                "Short toolbox talks (10 minutes) at shift start on rotating topics",
                "Assess understanding — a signature alone does not prove competence",
            ],
            "split-image-text",
            "Training is not a one-time event. Mention that untrained temporary staff "
            "during rush periods are a common weak link — brief them too.",
        ),
        _S(
            "Building a Food Safety Culture",
            [
                "Food safety is not a task on a checklist — it is a culture where every "
                "handler protects every patient.",
                "Leaders must model the behaviour: wash hands visibly, follow the rules, "
                "praise good practice.",
            ],
            "big-quote",
            "Pause here. Ask the group: 'What would you do if you saw a colleague skip "
            "handwashing during a rush?' Culture is what happens when nobody is watching.",
        ),
        _S(
            "Outbreak Response Plan",
            [
                "Stop service of the suspect food immediately and isolate/retain samples",
                "Notify the canteen manager, hospital infection-control team, and clinician",
                "Preserve evidence: keep samples refrigerated, secure all temperature logs",
                "Cooperate with the investigation; implement corrective actions and verify",
            ],
            "timeline",
            "Speed matters: the faster suspect food is isolated, the fewer people are "
            "exposed. Rehearse who calls whom — post the contact list in the kitchen.",
        ),
        _S(
            "Case Example: The Rice That Wasn't Cooled",
            [
                "A hospital canteen cooked a large batch of rice at noon and left it in a "
                "deep pot at room temperature until dinner service",
                "Bacillus cereus spores survived cooking and multiplied in the warm rice",
                "34 patients and staff developed vomiting within 6 hours of the meal",
                "Investigation found no cooling log and a 6-hour danger-zone exposure",
                "Lesson: cool rice fast in shallow trays, refrigerate within 2 hours, "
                "reheat once to steaming",
            ],
            "split-image-text",
            "This is a composite of real, well-documented outbreak patterns with cooked "
            "rice. Ask the group how their kitchen cools large batches today.",
        ),
        _S(
            "Common Violations to Avoid",
            [
                "✗ Bare-hand contact with ready-to-eat food",
                "✗ Cooked food held at room temperature 'until the rush'",
                "✗ Dirty wiping cloths used across raw and cooked areas",
                "✗ Pest sightings ignored or handled with unlabelled sprays",
                "✗ Expired or unlabelled ingredients kept 'just in case'",
```

```
                "✗ Temperature logs filled in from memory at shift end",
            ],
            "title-content",
            "Read each violation and ask for a show of hands: 'Seen this happen?' "
            "No blame — the goal is honest recognition, then fixing it.",
        ),
        _S(
            "Corrective Actions",
            [
                "Discard the affected batch and record the loss honestly",
                "Stop the line, clean and sanitise all contact surfaces",
                "Close the affected area and call the licensed pest operator",
                "Exclude the ill handler and review their recent tasks",
            ],
            "comparison",
            "Corrective action is not punishment — it is protection. Staff must feel safe "
            "reporting problems; hidden problems become outbreaks.",
            columns=["If This Happens", "Do This"],
        ),
        _S(
            "Preparing for an Audit",
            [
                "All records complete, signed, and filed for the required retention period",
                "Temperature logs current — no gaps, no back-filled entries",
                "Staff can explain their tasks: interview readiness beats memorised answers",
                "Premises walk-through: no clutter, labelled chemicals, intact screens",
                "Run a mock audit quarterly and close every finding with evidence",
            ],
            "checklist",
            "Auditors check three things: documents, premises, and people. If all three "
            "agree with each other, you pass.",
        ),
        _S(
            "Key Takeaways",
            [
                "Keep food out of the danger zone (5–60 °C): control time and temperature",
                "Separate raw from ready-to-eat — every time, no shortcuts",
                "Wash hands properly and report illness before handling food",
                "Clean first, then sanitise; document everything you do",
                "When in doubt, throw it out — patients trust us with their safety",
            ],
            "checklist",
            "Ask each participant to state one change they will make from tomorrow. "
            "End with: protecting patients starts in this kitchen.",
        ),
        _S(
            "References & Further Reading",
            [
                "World Health Organization — Five Keys to Safer Food (who.int/foodsafety)",
                "U.S. FDA Food Code — Chapters on temperature control and employee health",
                "ISO 22000:2018 — Food safety management systems requirements",
                "CDC — Foodborne illness surveillance and outbreak guidance (cdc.gov/foodsafety)",
                "Provincial food authority regulations and licensing requirements",
            ],
            "references",
            "Share these links with participants after the session for deeper reading.",
        ),
        _S(
            "Thank You",
            [],
            "thank-you",
            "Thank the participants for their attention and their daily work keeping "
            "patients safe. Open the floor for questions and discussion — the best "
            "learning often happens here.",
        ),
    ]


def build_sample_project() -> dict:
    """Build the complete 30-slide sample project (status 'ready')."""
```

```
    now = datetime.now().astimezone().isoformat()
    theme = get_theme("dark-green-gold")
    project = {
        "id": new_id("p_"),
        "title": _TITLE,
        "instructions": (
            "A 30-slide training deck for hospital canteen staff covering food safety "
            "hazards, temperature control, hygiene, HACCP, and incident response."
        ),
        "purpose": "Staff training workshop on food safety and hygiene",
        "audience": "Hospital canteen staff, food handlers, and supervisors",
        "knowledge_level": "Beginner to intermediate",
        "slide_count": 30,
        "theme": "dark-green-gold",
        "slide_size": "16:9",
        "provider": {"kind": "offline", "model": ""},
        "status": "ready",
        "outline": _outline(),
        "slides": [],
        "created": now,
        "updated": now,
    }
    # The final slide doubles as the Q&A prompt ("Questions & Discussion").
    project["slides"] = build_slides(project, project["outline"], theme)
    project["slide_count"] = len(project["slides"])
    return project
```

# Appendix B — Glossary and FAQ

## Glossary

| Term | Meaning |
|---|---|
| Canvas | The fixed 1000 x 562.5 coordinate space every slide is built on (16:9). |
| Element | One object on a slide: text, shape, table or image, with a stable id and geometry. |
| Layout | One of 13 named compositions that turn outline content into positioned elements. |
| Offline composer | The built-in deterministic outline builder that needs no AI provider or key. |
| Outline | The ordered list of slide titles, bullets, layout hints and notes you approve before building. |
| Provider | An optional AI backend: OpenAI, Gemini, Ollama — or Offline. |
| Theme | A named palette plus font pair; colours resolve at render time, so themes can be switched freely. |
| Validation | The named checks an exporter runs on its own file before a download is offered. |

## FAQ

### Do I need an API key?

No. Keep the provider on Offline and the built-in composer does the drafting. Add a key in Settings only if you want AI-written outlines, slide improvement or relayout.

### Is my presentation uploaded anywhere?

No. The app runs on your computer and stores projects as JSON under data/. Content leaves the machine only if you configure an online provider and use it.

### Can I edit the PowerPoint file afterwards?

Yes — that is the point of the PPTX export. Text boxes, shapes, tables and speaker notes are native PowerPoint objects, verified not to be screenshots.

### Why did my export fail validation?

Read the failing check in the report — for example a slide with no title text or a page-count mismatch. Fix that slide and export again; the Studio will not hand you a file it could not verify.

### Can it make charts, generate images, or redesign my old deck?

Not in v1 — those are roadmap items (Table 5). Images are labelled placeholders and the stats layout shows styled figures, not live charts.

### How do I back up or move my decks?

Copy the app's data/ folder. Projects are plain JSON files you can inspect in any text editor.

**— End — • Aesthetic AI Studio v1 • Creator: Dr. Asif Ahmad —**